

Master spécialisé

INGÉNIERIE INFORMATIQUE (M2I)

MÉMOIRE DU STAGE PROFESSIONNEL

Conception et implémentation full-stack du module Invoicing pour la plateforme KAFA'AT EMS



Présenté par :
ED-DAHMAN Y Redouane

Date de soutenance : 01 juillet 2026

Titre	Nom et Prénom	Établissement	Qualité
Pr.	Amjad Souad	Faculté des Sciences Tétouan	Président
Pr.	Rtayli Naoufal	Faculté des Sciences Tétouan	Examineur
Pr.	Benameur Lamyae	Faculté des Sciences Tétouan	Encadrant universitaire
Mr.	Saloumi Achraf	NTT DATA Tétouan	Encadrant professionnel

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

وَقَدْ عَلِمْنَا

صَدَقَ اللَّهُ الْعَظِيمُ

Dédicace

Je dédie ce travail à mes parents, qui ont été ma source constante d'inspiration, d'amour et de soutien tout au long de ma vie. Leur encouragement inconditionnel et leur confiance en moi ont été les fondements de ma réussite.

*Je tiens à exprimer ma profonde reconnaissance envers mes encadrants, le Professeur **Benameur Lamyae** et Mr. **Saloumi Achraf**, pour leur accompagnement, leurs conseils précieux et leur expertise tout au long de ce projet. Leur patience, leur disponibilité et leur dévouement ont été des moteurs essentiels de ma progression.*

J'adresse également mes remerciements à mes professeurs, qui m'ont transmis leur savoir, m'ont inspiré et ont élargi mes horizons intellectuels. Leurs enseignements ont façonné ma pensée critique et ont contribué à mon développement personnel et académique.

Enfin, je souhaite exprimer ma gratitude envers mes amis, qui ont été mes compagnons de route tout au long de cette aventure. Leur soutien moral, leur encouragement et leur amitié indéfectible m'ont permis de surmonter les obstacles et de persévérer dans mes efforts.

À tous ceux qui ont joué un rôle important dans ma vie académique et professionnelle, je vous adresse mes sincères remerciements et ma profonde reconnaissance. Votre influence et votre soutien continus ont été inestimables et ont contribué à ma réussite.

ED-DAHMAN Y Redouane

Remerciements

Je tiens tout d'abord à exprimer ma gratitude à Allah pour m'avoir accordé la santé, la détermination et la persévérance nécessaires pour mener à bien ce travail de recherche.

*Ensuite, je tiens à remercier chaleureusement mon encadrante universitaire, le Professeur **Benameur Lamyae**, pour son soutien constant, ses précieux conseils et son expertise tout au long de ce projet. Sa disponibilité, sa patience et sa bienveillance ont été d'une aide inestimable.*

*Je souhaite également exprimer ma reconnaissance à Mr. **Saloumi Achraf**, mon encadrant professionnel au sein de NTT DATA, pour son accompagnement, son expertise et ses conseils avisés. Sa vision et son expérience ont grandement contribué à la réussite de ce projet.*

*Je tiens à exprimer ma reconnaissance envers l'ensemble de l'équipe de **NTT DATA Maroc**, et tout particulièrement à Monsieur **Fred SABBAH**, Directeur Général, pour m'avoir offert l'opportunité de réaliser ce stage dans un environnement professionnel stimulant. Leur confiance en mes capacités et leur soutien ont été essentiels pour l'accomplissement de ce travail.*

*Je souhaite également remercier les membres du jury, le Professeur **Amjad Souad** et le Professeur **Rtayli Naoufal**, d'avoir accepté d'évaluer ce travail.*

Enfin, je tiens à exprimer ma gratitude envers ma famille et mes amis pour leur soutien indéfectible, leurs encouragements et leurs précieux conseils tout au long de ce parcours académique.

Mes sincères remerciements à toutes les personnes qui ont contribué de près ou de loin à la réalisation de ce travail.

Résumé

Ce rapport présente une synthèse du travail effectué dans le cadre de mon projet de fin d'études au sein de la société **NTT DATA** à Tétouan, dans le but d'obtenir mon diplôme du Master spécialisé en Ingénierie Informatique (M2I) de la Faculté des Sciences de Tétouan / Université Abdelmalek Essaâdi.

Ce projet s'est déroulé au sein d'une équipe de développement travaillant sur une plateforme d'entreprise à grande échelle. L'objectif principal de ce projet était de concevoir et d'implémenter de bout en bout (Full-Stack) le module complexe de facturation (**Invoicing**) pour le système de gestion des employés **KAFA'AT EMS**.

En raison de sa complexité et de sa logique métier pointue, ce module a été traité comme un microservice indépendant. Il prend en charge la gestion financière, les calculs liés aux temps de travail (Worktime overrides), l'application des règles de facturation, et la génération des factures.

L'application a été développée en utilisant le framework **Angular 17+** pour la partie frontend, garantissant une interface utilisateur réactive et ergonomique. Le backend a été construit avec **Java** et **Spring Boot 3.x**, en utilisant **PostgreSQL** pour la gestion et la persistance des données relationnelles.

La méthodologie adoptée pour la réalisation de ce projet s'est appuyée sur les pratiques agiles, allant de l'étude et la spécification des besoins à la modélisation de l'architecture et des fonctionnalités via le langage UML, pour aboutir à l'intégration et au déploiement du module.

Mots clés :

Application Web, Plateforme EMS, Invoicing, Microservice, NTT DATA, Angular, Spring Boot, PostgreSQL, Full-Stack.

Abstract

This report presents a summary of the work carried out as part of my graduation project (Master's thesis) at **NTT DATA** in Tetouan, in order to obtain my specialized Master's degree in Computer Engineering (M2I) from the Faculty of Sciences in Tetouan / Abdelmalek Essaâdi University.

This project took place within a development team working on a large-scale enterprise platform. The main objective of this project was to design and implement from scratch (Full-Stack) the complex **Invoicing** module for the **KAFA'AT** Employee Management System (EMS).

Due to its complexity and advanced business logic, this module was treated as an independent microservice. It handles financial management, calculations related to worktime overrides, the application of billing rules, and invoice generation.

The application was developed using the **Angular 17+** framework for the frontend, ensuring a responsive and ergonomic user interface. The backend was built with **Java** and **Spring Boot 3.x**, using **PostgreSQL** for relational data management and persistence.

The methodology adopted for the realization of this project was based on agile practices, ranging from the study and specification of requirements to the modeling of the architecture and functionalities using UML language, culminating in the integration and deployment of the module.

Keywords :

Web Application, EMS Platform, Invoicing, Microservice, NTT DATA, Angular, Spring Boot, PostgreSQL, Full-Stack.

Table des matières

Dédicace	III
Remerciements	IV
Résumé	V
Abstract	VI
Table des figures	X
Liste des tableaux	XII
Liste des abréviations	XIII
Introduction générale	1
1 Contexte général du projet	3
Introduction	3
1.1 Présentation de l'organisme d'accueil	3
1.1.1 Présentation générale de NTT DATA	3
1.1.2 Historique NTT DATA	3
1.1.3 Fiche technique	4
1.1.4 Couverture mondiale NTT DATA	5
1.1.5 Secteur d'activités	5
1.1.6 Services offerts par NTT DATA	6
1.1.7 Partenaires stratégiques de NTT DATA	7
1.1.8 Clients de NTT DATA	7
1.1.9 Technologie utilisée par NTT DATA	8
1.1.10 NTT DATA Maroc	9
1.1.11 Organigramme de NTT DATA Maroc	9
1.1.12 Modèle de carrière	10
1.2 Cadre général du projet	10
1.2.1 Contexte du projet	10
1.2.2 Problématique	11
1.2.3 Solution proposée	11
1.2.4 Cible du projet	12
1.2.5 Fonctionnalités	12
1.2.5.1 Besoins fonctionnels	12
1.2.5.2 Besoins non-fonctionnels	13
1.3 Conduite et planification du projet	14
1.3.1 Processus de développement	14

1.3.2	La méthode SCRUM	14
1.3.2.1	Pourquoi SCRUM ?	15
1.3.2.2	Les principes fondamentaux de SCRUM	15
1.3.2.3	Équipe du projet	16
1.3.2.4	Gestion de projet avec JIRA	17
1.3.2.5	Détail des responsabilités de l'équipe	18
1.3.2.6	Organisation et communication (Microsoft Teams)	20
1.3.3	Planification temporelle	21
1.3.3.1	Diagramme de GANTT	22
1.3.3.2	Découpage en Sprints	22
	Conclusion	24
2	Analyse et Conception	26
	Introduction	26
2.1	Langage UML	26
2.1.1	Définition	26
2.1.2	Intérêts de la modélisation UML	27
2.2	Modélisation du système	27
2.2.1	Diagramme de cas d'utilisation	27
2.2.1.1	Identification des acteurs	28
2.2.1.2	Diagramme de cas d'utilisation général de l'application	28
2.2.1.3	Diagramme de cas d'utilisation de gestion du temps de travail	29
2.2.1.4	Diagramme de cas d'utilisation de gestion de la facturation	30
2.2.1.5	Diagramme de cas d'utilisation de gestion des groupes et tarification	31
2.2.1.6	Diagramme de cas d'utilisation de gestion des utilisateurs	32
2.2.2	Diagramme de classes	33
2.2.3	Diagramme de séquence	35
2.2.3.1	Diagramme de séquence « S'authentifier »	36
2.2.3.2	Diagramme de séquence « Soumettre un pointage »	37
2.2.3.3	Diagramme de séquence « Génération massive des factures »	38
2.2.3.4	Diagramme de séquence « Désactiver un compte utilisateur »	39
2.2.3.5	Diagramme de séquence « Créer un groupe de facturation »	40
2.2.3.6	Diagramme de séquence « Appliquer un ajustement »	41
2.2.3.7	Diagramme de séquence « Gestion détaillée d'une facture »	42
2.2.4	Diagrammes d'activité	44
2.2.4.1	Diagramme d'activité « Validation des pointages »	44
2.2.4.2	Diagramme d'activité « Génération Massive des Factures »	45
2.2.4.3	Diagramme d'activité « Suivi des factures »	46
	Conclusion	48
3	Réalisation et Mise en œuvre	50
	Introduction	50
3.1	Architecture de l'application	50
3.1.1	Architecture Backend	51
3.1.2	Architecture Frontend	52
3.2	Technologies et outils de développement	53
3.2.1	Environnements logiciels	53

3.2.1.1	IntelliJ IDEA	53
3.2.1.2	Visual Studio Code	53
3.2.1.3	Swagger (OpenAPI)	53
3.2.1.4	Apache Tomcat	54
3.2.1.5	Docker	54
3.2.1.6	Apache Maven	55
3.2.1.7	Node.js	55
3.2.1.8	NPM (Node Package Manager)	55
3.2.1.9	JWT (JSON Web Token)	56
3.2.1.10	Postman	56
3.2.1.11	PlantUML	57
3.2.2	Frameworks et technologies clés	57
3.2.2.1	Spring Boot	57
3.2.2.2	Spring Security	57
3.2.2.3	Spring Data JPA	58
3.2.2.4	Angular	58
3.2.2.5	Sass (SCSS)	58
3.2.3	Frameworks de tests et Assurance Qualité	59
3.2.3.1	JUnit	59
3.2.3.2	Mockito	59
3.2.3.3	Jest	59
3.2.4	Outils de gestion de projet et de collaboration	60
3.2.4.1	Jira Software	60
3.2.4.2	Confluence	60
3.2.5	Contrôle de version et collaboration	60
3.2.5.1	Git	60
3.2.5.2	GitLab	61
3.3	Présentation des interfaces de l'application	61

Table des figures

Figure 1.1 : Logo officiel de l'organisme d'accueil NTT DATA	3
Figure 1.2 : Historique NTT DATA	4
Figure 1.3 : Couverture mondiale NTT DATA	5
Figure 1.4 : Quelques secteurs d'activités de NTT DATA	6
Figure 1.5 : NTT DATA Services	6
Figure 1.6 : Partenaires stratégiques de NTT DATA	7
Figure 1.7 : Clients de NTT DATA	7
Figure 1.8 : Technologie utilisée par NTT DATA	8
Figure 1.9 : Organigramme de NTT DATA Maroc	9
Figure 1.10 : Modèle de carrière	10
Figure 1.11 : La méthode SCRUM	15
Figure 1.12 : L'équipe SCRUM	16
Figure 1.13 : Interface de gestion de projet JIRA	17
Figure 1.14 : Exemple de suivi de l'avancement des tâches sur JIRA	18
Figure 1.15 : Plateforme de communication Microsoft Teams	20
Figure 1.16 : Collaboration de l'équipe sur Microsoft Teams	21
Figure 1.17 : Diagramme de GANTT du projet Invoicing	22
Figure 2.1 : Logo officiel du langage UML	26
Figure 2.2 : Diagramme de cas d'utilisation général du module Invoicing	29
Figure 2.3 : Diagramme de cas d'utilisation de gestion du temps de travail	30
Figure 2.4 : Diagramme de cas d'utilisation de gestion de la facturation	31
Figure 2.5 : Diagramme de cas d'utilisation de gestion des groupes et tarification	32
Figure 2.6 : Diagramme de cas d'utilisation de gestion des utilisateurs et des accès	33
Figure 2.7 : Diagramme de classes du module Invoicing et Worktime	35
Figure 2.8 : Diagramme de séquence d'authentification (JWT & Spring Security)	36
Figure 2.9 : Diagramme de séquence de la soumission d'un pointage	37
Figure 2.10 : Diagramme de séquence de la génération massive des factures	38
Figure 2.11 : Diagramme de séquence de la désactivation d'un compte (Soft Delete)	39
Figure 2.12 : Diagramme de séquence de la création d'un groupe de facturation	40
Figure 2.13 : Diagramme de séquence d'un ajustement de pointage (Override)	41
Figure 2.14 : Diagramme de séquence de la gestion détaillée d'une facture	43
Figure 2.15 : Diagramme d'activité avec Swimlanes : Validation des pointages	45
Figure 2.16 : Diagramme d'activité détaillé : Moteur de génération massive des factures	46
Figure 2.17 : Diagramme d'activité : Gestion, relances et recouvrement des factures	47
Figure 3.1 : Architecture globale de l'application	50
Figure 3.2 : Architecture en couches du Backend (Spring Boot)	51
Figure 3.3 : Architecture interne de l'application cliente (Angular)	52
Figure 3.4 : Logo d'IntelliJ IDEA	53

Figure 3.5 : Logo de Visual Studio Code	53
Figure 3.6 : Logo de Swagger	53
Figure 3.7 : Logo d'Apache Tomcat	54
Figure 3.8 : Logo de Docker	54
Figure 3.9 : Logo d'Apache Maven	55
Figure 3.10 : Logo de Node.js	55
Figure 3.11 : Logo de NPM	55
Figure 3.12 : Logo de JWT	56
Figure 3.13 : Logo de Postman	56
Figure 3.14 : Logo de PlantUML	57
Figure 3.15 : Logo de Spring Boot	57
Figure 3.16 : Logo de Spring Security	57
Figure 3.17 : Logo de Spring Data JPA	58
Figure 3.18 : Logo d'Angular	58
Figure 3.19 : Logo de Sass	58
Figure 3.20 : Logo de JUnit	59
Figure 3.21 : Logo de Mockito	59
Figure 3.22 : Logo de Jest	59
Figure 3.23 : Logo de Jira Software	60
Figure 3.24 : Logo de Confluence	60
Figure 3.25 : Logo de Git	60
Figure 3.26 : Logo de GitLab	61

Liste des tableaux

Tableau 1.1 : Fiche technique globale et locale de NTT DATA	4
Tableau 1.2 : Détail approfondi des modules et tâches par membre de l'équipe	19
Tableau 1.3 : Description globale et temporelle des Sprints	23

Liste des abréviations

Abréviation	Signification
API	Application Programming Interface (Interface de Programmation Applicative)
CI/CD	Continuous Integration / Continuous Deployment (Intégration et Déploiement Continus)
DAO	Data Access Object (Objet d'Accès aux Données)
DI	Dependency Injection (Injection de Dépendances)
DTO	Data Transfer Object (Objet de Transfert de Données)
EMS	Employee Management System (Système de Gestion des Employés)
HTTP	HyperText Transfer Protocol (Protocole de Transfert Hypertexte)
IDE	Integrated Development Environment (Environnement de Développement Intégré)
IoC	Inversion of Control (Inversion de Contrôle)
JPA	Java Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token (Jeton Web JSON)
MVC	Model-View-Controller (Modèle-Vue-Contrôleur)
ORM	Object-Relational Mapping (Mappage Objet-Relationnel)
REST	Representational State Transfer
SGBDR	Système de Gestion de Base de Données Relationnelle (ex : PostgreSQL)
SPA	Single Page Application (Application Web Monopage)
SQL	Structured Query Language (Langage de Requête Structurée)
UML	Unified Modeling Language (Langage de Modélisation Unifié)

Introduction générale

La transformation digitale et l'optimisation continue des processus de gestion interne sont devenues des enjeux majeurs pour les entreprises cherchant à maintenir leur compétitivité. Dans cette optique, **NTT DATA**, leader mondial des services informatiques, s'engage à innover et à moderniser ses propres systèmes d'information. C'est dans ce contexte que s'inscrit notre projet de fin d'études, intitulé :

« Conception et implémentation full-stack du module Invoicing pour la plateforme KAFA'AT EMS ».

Ce projet, réalisé au sein de **NTT DATA Tétouan**, a pour but de concevoir et développer de bout en bout un nouveau microservice indépendant dédié à la facturation (**Invoicing**). Ce module a pour objectif d'automatiser la gestion financière, de traiter les calculs liés aux temps de travail, d'appliquer les règles de facturation, et de générer les factures de manière automatisée, fiable et performante.

Afin de détailler les différentes phases de réalisation de ce projet, le présent rapport est articulé autour de trois chapitres principaux :

Chapitre 1

Introduit l'organisme d'accueil (**NTT DATA**), le contexte général du projet pour la plateforme **KAFA'AT EMS**, et détaille la méthodologie de travail adoptée (Agile Scrum).

Chapitre 2

Consacré à la spécification des besoins, à l'étude fonctionnelle et à la **conception** technique. Il détaille les choix d'architecture orientée **Microservices** et la modélisation **UML** du système.

Chapitre 3

Se concentre sur la **réalisation**. Il détaille l'environnement technologique (**Angular**, **Spring Boot**, **PostgreSQL**) et présente les interfaces réalisées.

Ce rapport offre ainsi une vue d'ensemble complète du cycle de vie du développement de ce module, en mettant en évidence les défis techniques relevés et la valeur ajoutée apportée au système global de l'entreprise.

Chapitre 1

Contexte général du projet

Introduction

Dans ce premier chapitre, nous allons situer notre projet de fin d'études dans son contexte général. Nous commencerons par présenter l'organisme d'accueil, **NTT DATA**, au sein duquel ce travail a été réalisé. Ensuite, nous définirons le cadre du projet en détaillant la problématique, la solution proposée pour le module **Invoicing**, et les objectifs visés. Enfin, nous aborderons la méthodologie de travail mise en place pour assurer le bon déroulement et la planification du projet.

1.1 Présentation de l'organisme d'accueil

1.1.1 Présentation générale de NTT DATA

NTT DATA est une entreprise multinationale de services informatiques et de conseil, faisant partie du groupe NTT (*Nippon Telegraph and Telephone Corporation*), l'un des plus grands conglomérats de télécommunications au monde. Fondée au Japon en 1967, NTT DATA fournit une gamme complète de services informatiques et de solutions technologiques à ses clients à travers le monde.

En tant que fournisseur global, l'entreprise propose une large gamme de services pour aider ses clients à transformer leurs activités. Ces services comprennent le conseil en stratégie, l'intégration de systèmes, le développement et la maintenance d'applications, la gestion de l'infrastructure, ainsi que des services liés au *cloud computing* et à l'analyse de données.

NTT DATA se positionne comme un partenaire stratégique pour ses clients, en leur fournissant des solutions technologiques innovantes basées sur des technologies émergentes. L'entreprise s'appuie sur son expertise technique et son expérience mondiale pour aider ses clients à améliorer leur efficacité opérationnelle, innover et rester compétitifs sur le marché.



FIGURE 1.1 – Logo officiel de l'organisme d'accueil NTT DATA

1.1.2 Historique NTT DATA

Avant de plonger dans l'historique détaillé de NTT DATA, il convient de noter que l'entreprise a connu une croissance et une évolution significatives depuis sa création en 1967. De ses modestes débuts en tant que filiale de NTT, NTT DATA est devenue une entreprise mondiale de premier

plan dans le secteur des services informatiques et du conseil. Voici une figure des étapes clés de son parcours historique :

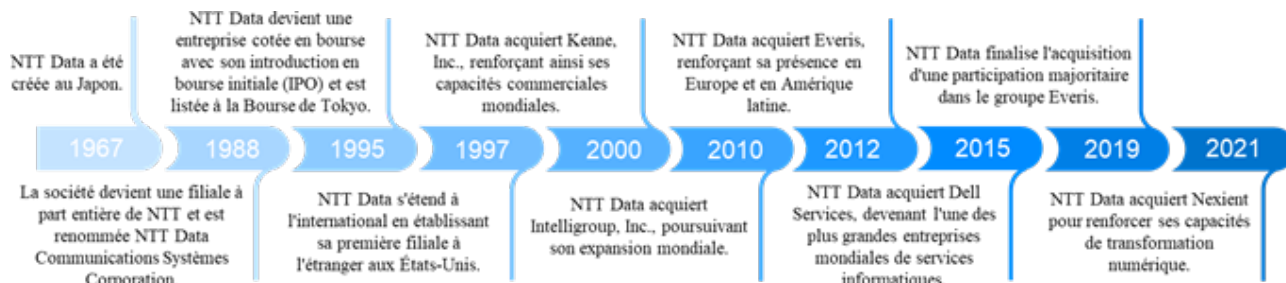


FIGURE 1.2 – Historique NTT DATA

Aujourd'hui, NTT DATA est reconnue comme l'un des leaders mondiaux des services informatiques et du conseil, offrant des solutions technologiques innovantes à ses clients dans le monde entier.

1.1.3 Fiche technique

Le tableau ci-dessous résume les informations générales et juridiques de l'entreprise :

Caractéristique	Information
Raison Sociale / Nom	NTT DATA
Forme Juridique	Global : Société commerciale / Société civile Maroc : Société à Responsabilité Limitée (SARL)
Date de Création	1 ^{er} juillet 1988
Directeur Général	Global : Yo Honma Maroc : Fred Sabbah
Siège Social & Adresse	Global : Tokyo, Japan Maroc : Parc de Tetouanshore, Route de Cabo Negro - Martil
Taille de l'entreprise	Grande
Activités	Services informatiques mondiaux, développement de logiciels, gestion des systèmes d'information, externalisation des processus métier. L'étude, la réalisation et la mise en œuvre de solutions d'automatisation pour le secteur privé et les administrations publiques.

TABLE 1.1 – Fiche technique globale et locale de NTT DATA

1.1.4 Couverture mondiale NTT DATA

NTT DATA possède une vaste couverture mondiale avec des bureaux et des centres de prestation de services dans de nombreux pays à travers le monde.

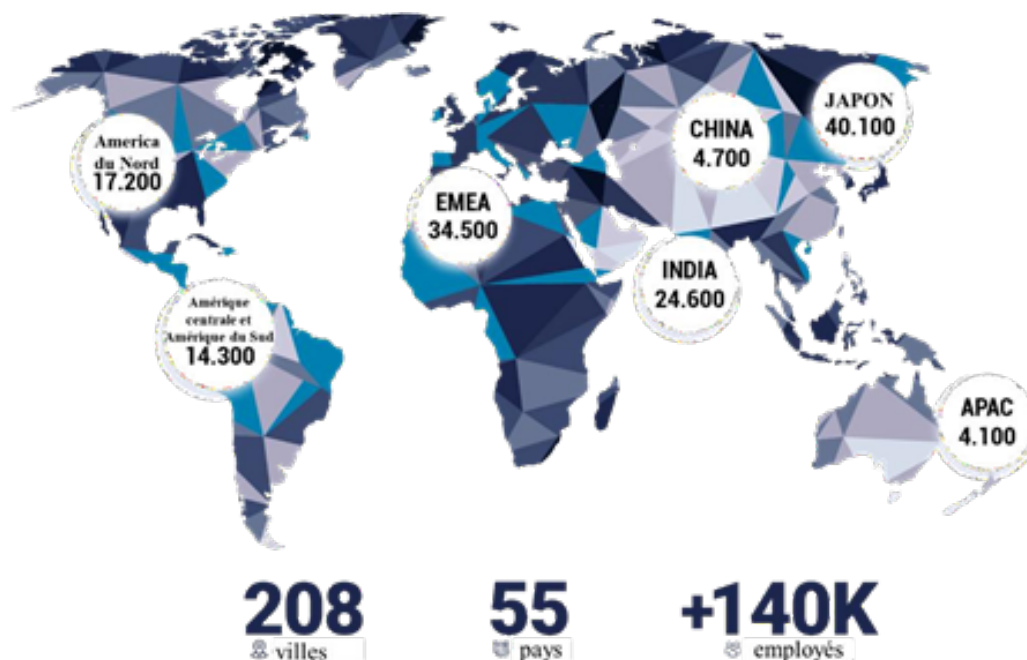


FIGURE 1.3 – Couverture mondiale NTT DATA

1.1.5 Secteur d'activités

NTT DATA opère dans le secteur des services informatiques et du conseil. L'entreprise propose une gamme complète de services et de solutions technologiques pour aider ses clients à résoudre leurs défis informatiques et à atteindre leurs objectifs commerciaux. Voici quelques-uns des principaux secteurs d'activité dans lesquels NTT DATA est impliquée :

- **Services financiers** : NTT DATA travaille avec des institutions financières, des banques, des compagnies d'assurance et d'autres acteurs du secteur financier pour fournir des solutions informatiques et des services de conseil qui améliorent l'efficacité opérationnelle, la gestion des risques, l'expérience client et la conformité réglementaire.
- **Santé et sciences de la vie** : NTT DATA fournit des solutions technologiques aux fournisseurs de soins de santé, aux compagnies pharmaceutiques et aux organismes de recherche médicale. Cela inclut la gestion des dossiers médicaux électroniques, les systèmes de gestion des soins de santé, l'analyse des données de santé, les services de télémédecine et les solutions de suivi des essais cliniques.
- **Secteur public** : NTT DATA travaille en étroite collaboration avec les gouvernements et les organismes du secteur public pour fournir des solutions informatiques et des services de conseil qui améliorent la prestation des services publics, la gestion des données, la sécurité publique, la gestion des ressources et l'efficacité administrative.

- **Secteur manufacturier et automobile** : NTT DATA offre des solutions technologiques aux fabricants et aux entreprises automobiles pour optimiser les opérations de fabrication, améliorer la chaîne d'approvisionnement, intégrer l'Internet des objets (IoT) dans les processus de production et développer des solutions de mobilité intelligente.
- **Télécommunications et médias** : NTT DATA collabore avec les fournisseurs de services de télécommunications et les entreprises médiatiques pour fournir des solutions d'infrastructure informatique, des services de transformation numérique, des solutions d'analyse des données, des plateformes de services en ligne et des solutions de gestion des clients.

Outre ces secteurs spécifiques, NTT DATA travaille également avec des entreprises de diverses industries, telles que l'énergie, les transports, le commerce de détail, l'industrie du voyage et de l'hôtellerie, pour fournir des services informatiques et des solutions personnalisées adaptées à leurs besoins spécifiques. [3]

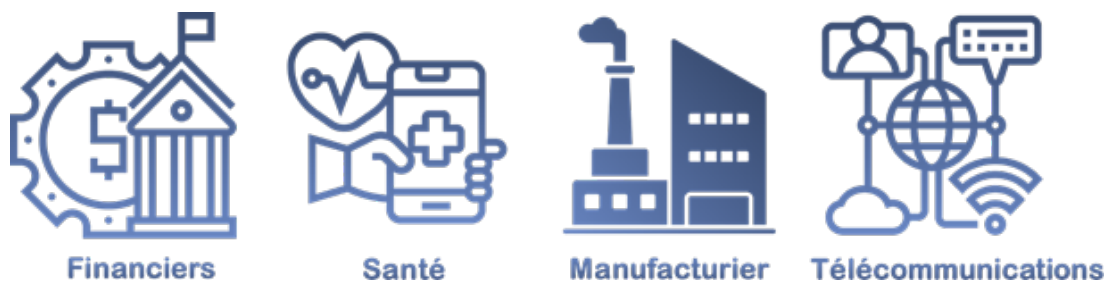


FIGURE 1.4 – Quelques secteurs d'activités de NTT DATA

1.1.6 Services offerts par NTT DATA

NTT DATA, en tant que fournisseur mondial de services informatiques de confiance, propose une gamme complète de services pour aider les entreprises à relever les défis numériques et à innover. La figure suivante montre les services offerts par société NTT DATA [4]



FIGURE 1.5 – NTT DATA Services

1.1.7 Partenaires stratégiques de NTT DATA

NTT DATA a établi des partenariats stratégiques avec de nombreuses entreprises leaders dans le domaine de la technologie et des services. Ces partenariats sont essentiels pour renforcer l'offre de solutions de NTT DATA et offrir une valeur ajoutée à ses clients. Parmi les partenaires clés figurent des géants de l'industrie tels que Microsoft, Oracle, SAP, Salesforce, IBM et Amazon Web Services (AWS). Ces partenariats permettent à NTT DATA d'accéder aux dernières technologies et d'offrir des solutions innovantes dans des domaines tels que le cloud computing, les logiciels d'entreprise, la gestion de la relation client, l'analyse des données et bien plus encore. Grâce à ces partenariats stratégiques, NTT DATA peut offrir des services de conseil, d'intégration et de gestion des solutions basées sur les plateformes et les technologies de ses partenaires, répondant ainsi aux besoins variés de ses clients dans divers secteurs d'activité. [5]



FIGURE 1.6 – Partenaires stratégiques de NTT DATA

1.1.8 Clients de NTT DATA

Les clients de NTT DATA comprennent une vaste gamme d'entreprises et d'organisations à travers le monde. Grâce à son expertise technologique et à ses solutions innovantes, NTT DATA a su établir des partenariats durables avec des clients dans divers secteurs tels que la finance, les télécommunications, la santé, la fabrication et bien d'autres. Que ce soit pour des grandes entreprises internationales ou des entreprises locales, NTT DATA s'engage à fournir des services de haute qualité, adaptés aux besoins spécifiques de chaque client. L'objectif principal est de créer des relations de confiance à long terme et de contribuer à la réussite et à la croissance de ses clients. Voici quelques-uns des clients principaux de NTT DATA : [3]

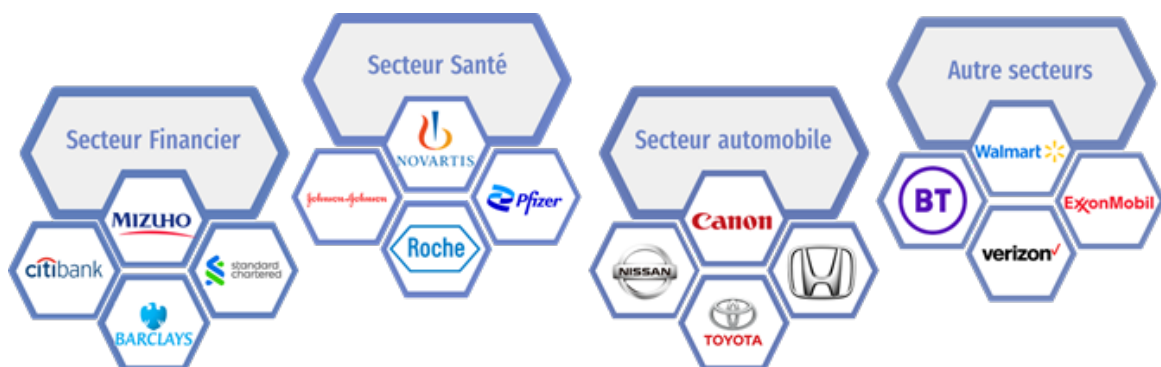


FIGURE 1.7 – Clients de NTT DATA

1.1.9 Technologie utilisée par NTT DATA

NTT DATA utilise un large éventail de technologies pour exécuter et faire évoluer les services d'application pour ses clients. Parmi les technologies clés utilisées figurent Java, .NET, SQL, Cobol, le testing (tests logiciels) ainsi que la RPA (Robotic Process Automation). Ces technologies sont mises en œuvre par une équipe hautement qualifiée et talentueuse, spécialisée non seulement dans les aspects techniques, mais aussi dans les connaissances sectorielles. En combinant ces technologies de pointe avec une expertise sectorielle approfondie, NTT DATA est en mesure de fournir des solutions innovantes et adaptées aux besoins spécifiques de ses clients, tout en garantissant une qualité élevée et une productivité optimale dans la réalisation de leurs projets d'application. [6]

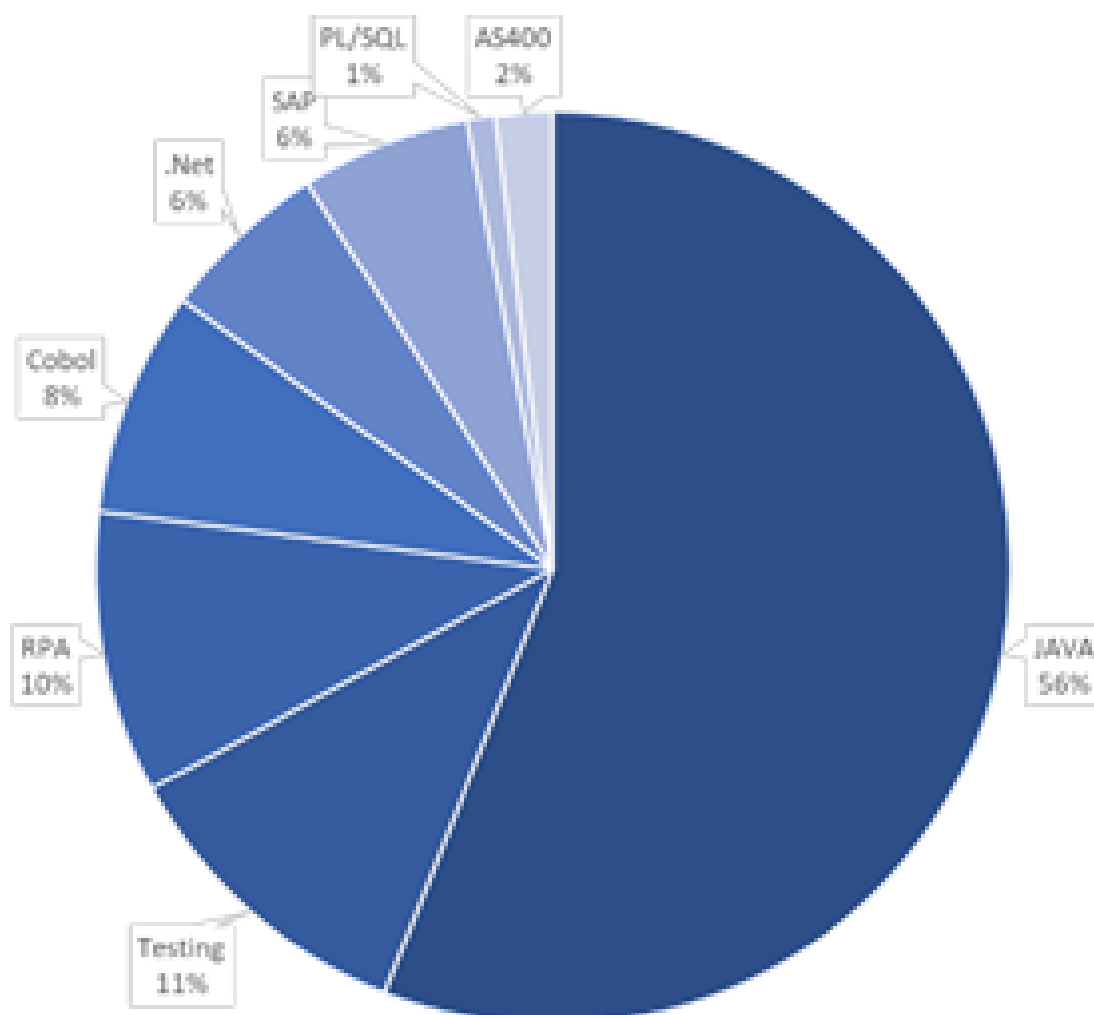


FIGURE 1.8 – Technologie utilisée par NTT DATA

1.1.10 NTT DATA Maroc

NTT DATA est également présent au Maroc. NTT DATA Maroc est une filiale de NTT DATA Corporation, fournisseur mondial de services informatiques et de solutions technologiques. L'entreprise offre une gamme complète de services, notamment le développement de logiciels, la gestion des infrastructures informatiques, les services de conseil en technologie, la transformation numérique et bien plus encore.

NTT DATA Maroc travaille avec des clients locaux et internationaux dans divers secteurs d'activité tels que la finance, les télécommunications, la santé, l'industrie manufacturière, le secteur public, le commerce de détail et d'autres industries. L'objectif de NTT DATA Maroc est d'aider les entreprises à optimiser leurs opérations, à améliorer leur efficacité et à relever les défis technologiques auxquels elles sont confrontées.

En tant que filiale de NTT DATA Corporation, NTT DATA Maroc bénéficie de l'expertise et de l'expérience mondiale de l'entreprise dans le domaine des technologies de l'information. Cela lui permet de proposer des solutions de pointe et d'innover dans le domaine des services informatiques pour répondre aux besoins spécifiques du marché marocain.

NTT DATA Maroc contribue également à l'écosystème technologique du pays en collaborant avec des partenaires locaux, des universités et des institutions pour promouvoir l'innovation, le développement des compétences et la croissance du secteur des technologies de l'information au Maroc. [7]

1.1.11 Organigramme de NTT DATA Maroc

L'organigramme de NTT DATA Maroc offre une représentation visuelle de la structure interne de l'entreprise, mettant en évidence les départements, les responsabilités et les relations hiérarchiques qui existent au sein de l'organisation.

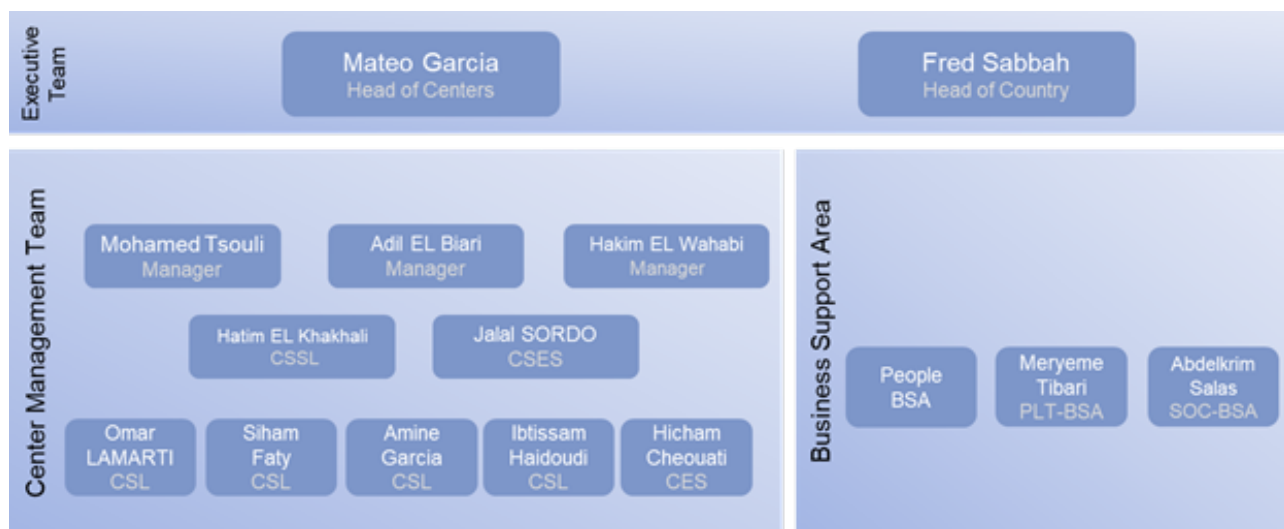


FIGURE 1.9 – Organigramme de NTT DATA Maroc

1.1.12 Modèle de carrière

NTT DATA utilise un modèle de carrière pour permettre aux employés de développer leurs compétences, de progresser professionnellement et d'atteindre leurs objectifs professionnels au sein de l'entreprise.

Le modèle de carrière de NTT DATA se caractérise par les aspects suivants :

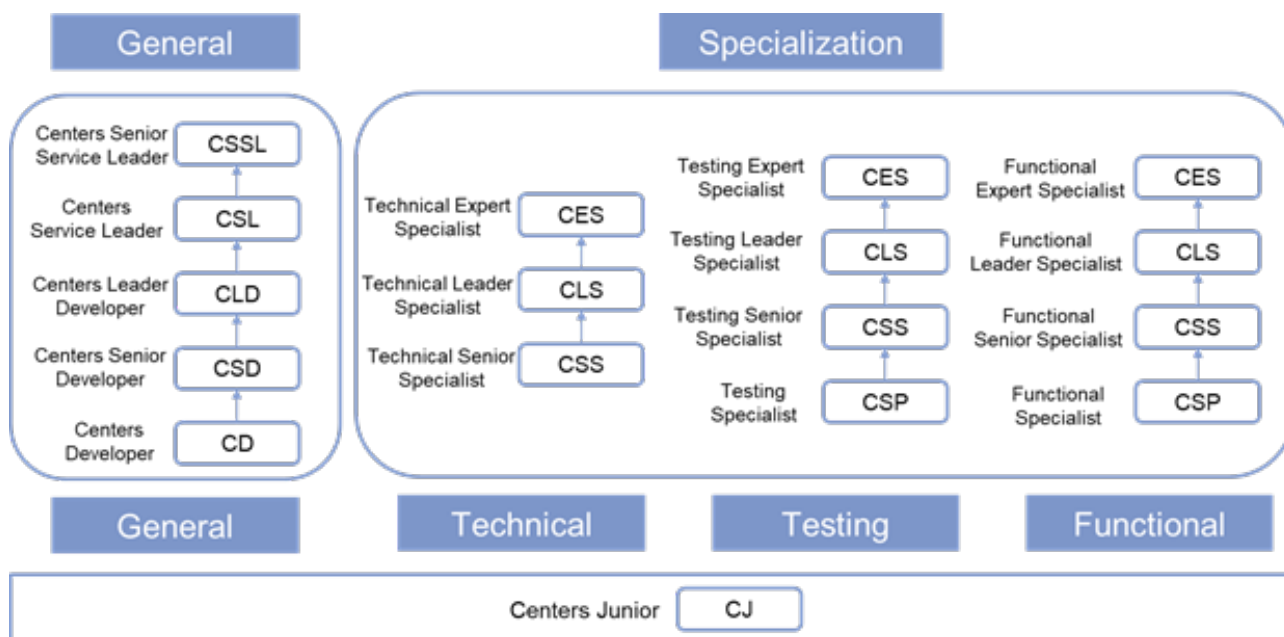


FIGURE 1.10 – Modèle de carrière

1.2 Cadre général du projet

1.2.1 Contexte du projet

Ce projet s'inscrit dans le cadre de la modernisation et de l'optimisation continue des systèmes d'information internes de l'entreprise. NTT DATA dispose d'une plateforme globale et centralisée pour la gestion de ses collaborateurs nommée **KAFA'AT EMS** (Employee Management System).

Afin d'étendre les capacités de cette plateforme et de répondre aux exigences croissantes en matière de gestion financière, l'entreprise souhaite concevoir et développer de zéro un nouveau module complet et indépendant dédié à la facturation : le module **Invoicing**. Ce projet, réalisé au sein des équipes de développement de NTT DATA Tétouan, vise à doter la plateforme EMS d'un outil puissant pour automatiser et fiabiliser le cycle de facturation basé sur les temps de travail et les prestations des employés.

1.2.2 Problématique

La gestion de la facturation dans une multinationale de l'envergure de NTT DATA est un processus complexe. Elle implique le traitement d'une quantité massive de données croisées : les profils des employés, les assignations aux projets, les taux journaliers, et surtout les temps de travail réels ainsi que leurs ajustements (Worktime overrides).

Actuellement, l'absence d'un module centralisé et automatisé intégré directement à la plateforme EMS peut entraîner plusieurs défis. Les processus manuels ou l'utilisation d'outils fragmentés augmentent considérablement le risque d'erreurs de calcul, créent des goulots d'étranglement administratifs et provoquent des retards dans l'émission des factures. De plus, l'application manuelle de règles de facturation spécifiques à chaque client ou projet est fastidieuse et rend difficile le suivi financier en temps réel.

La problématique majeure réside donc dans la conception d'une solution technique robuste capable de surmonter ces défis. Comment concevoir et implémenter un microservice Full-Stack évolutif qui s'intègre harmonieusement avec l'écosystème KAFA'AT EMS existant, tout en garantissant un calcul exact des données financières et en offrant aux gestionnaires une interface fluide, réactive et sécurisée pour la génération des factures ?

Cependant, dans le cadre de la gestion financière actuelle, les équipes chargées de la facturation rencontrent souvent des difficultés liées à la manipulation de fichiers Excel complexes et à la fragmentation des données. En particulier, lorsque le nombre de collaborateurs et de projets est important, il devient extrêmement difficile de consolider les heures travaillées (Worktime) et d'appliquer les ajustements de manière précise et rapide sans risquer l'erreur humaine.

De plus, le volume des données à traiter à la fin de chaque cycle de facturation nécessite une attention particulière et un temps considérable. À ce stade, il est primordial de trouver une solution technique qui facilite le travail des gestionnaires de manière efficace et moderne, en simplifiant les calculs complexes. Cette solution devrait permettre de générer des factures précises et conformes aux règles métier en quelques clics seulement.

1.2.3 Solution proposée

Pour remédier à ces problèmes et optimiser le processus de gestion financière, la solution proposée consiste en la conception et la mise en place d'un microservice innovant dédié exclusivement à la facturation : le module **Invoicing**. Cette application Full-Stack viendra s'intégrer nativement à la plateforme globale KAFA'AT EMS.

Ce nouveau module permettrait aux responsables de centraliser toutes les informations financières, de mettre à jour les données de facturation en temps réel et de gérer efficacement les règles de calcul liées aux temps de travail. Il offrirait également la possibilité d'automatiser entièrement la génération des factures, réduisant ainsi drastiquement la charge de travail manuelle et les risques d'erreurs.

Grâce à cette solution, le suivi des prestations et des ajustements horaires (Worktime overrides) serait grandement simplifié. Les gestionnaires pourraient facilement ajouter, modifier et consulter les informations pertinentes pour chaque projet, même lorsque le volume des données est élevé. De plus, l'architecture choisie (**Angular** pour le frontend et **Spring Boot** pour le backend) garantira une application web hautement réactive, évolutive et facile à maintenir.

En résumé, la mise en place de ce module d'Invoicing permettrait à NTT DATA de moderniser et d'optimiser ses processus internes de gestion financière au sein de l'EMS. Cela facilitera non seulement le suivi précis des factures, mais également la prise de décision stratégique grâce à une vue d'ensemble claire sur les données financières des projets.

1.2.4 Cible du projet

Le module d'Invoicing s'adresse principalement aux chefs de projet (Project Managers), aux responsables de livraison (Service Delivery Managers) et à l'équipe financière au sein de NTT DATA. Ces acteurs clés sont responsables de la gestion financière des projets et du suivi de la facturation. L'application leur fournira les outils nécessaires pour suivre les temps de travail en temps réel et gérer efficacement l'émission des factures. Elle leur permettra également de simplifier le processus de validation des heures (Worktime overrides), en facilitant l'ajout et la consultation des données financières pertinentes pour chaque projet, même lorsque le volume des données est élevé. Grâce à ce module, les équipes pourront coordonner et organiser efficacement les différentes étapes du cycle de facturation au sein de la plateforme KAFA'AT EMS.

1.2.5 Fonctionnalités

1.2.5.1 Besoins fonctionnels

Voici une liste spécifique des exigences fonctionnelles majeures pour le module Invoicing de la plateforme KAFA'AT EMS :

❖ Authentification et gestion des utilisateurs :

- Les utilisateurs doivent pouvoir s'authentifier de manière sécurisée avec des identifiants uniques.
- Le système doit permettre la création, la modification et la désactivation de comptes utilisateur.
- Différents niveaux d'accès (Rôles) doivent être définis (ex : Administrateur, Manager, Financier) pour contrôler les actions et les vues disponibles pour chaque utilisateur.

❖ Gestion de la facturation (Invoicing) :

- Le système doit permettre la génération, la modification et l'annulation des factures liées aux projets.
- Les informations clés telles que le numéro de facture, le montant calculé, la date d'émission et le statut (Payée, En attente) doivent être enregistrées.

- Les utilisateurs doivent pouvoir consulter la liste des factures, les filtrer (par client, par date, par statut) et effectuer des recherches avancées.

❖ Gestion des temps de travail (Worktime Overrides) :

- Le système doit permettre l'ajout, la modification et la validation des ajustements d'heures travaillées pour chaque collaborateur.
- Les données croisées telles que les taux journaliers (TJM), le projet affecté et le nombre d'heures réelles doivent être gérées avec précision pour alimenter le calcul automatique des factures.

❖ Gestion des règles de facturation et des clients :

- Le système doit permettre l'ajout, la modification et la suppression de profils clients ou de projets.
- Les détails financiers tels que la devise, les conditions de paiement et les règles de taxes doivent être configurables.
- Les coordonnées des responsables de la facturation (nom, numéro de téléphone, e-mail) doivent être enregistrées.
- Le système doit permettre la génération, le téléchargement et la gestion des factures au format PDF pour un envoi facilité aux clients.

❖ Tableau de bord et reporting financier :

- Le système doit fournir un tableau de bord (Dashboard) permettant aux gestionnaires de visualiser des statistiques et des informations clés sur les revenus et les factures.
- Les indicateurs de performance (KPIs) tels que le montant total facturé, le nombre de factures en attente de paiement ou en retard, doivent être présentés de manière claire et concise (graphiques, diagrammes).

Ces exigences fonctionnelles définissent les principales capacités de l'application de facturation (Invoicing) de NTT DATA, en se concentrant sur les besoins spécifiques des équipes financières et des chefs de projet.

1.2.5.2 Besoins non-fonctionnels

L'application à concevoir doit être conforme aux exigences non-fonctionnelles suivantes :

- **Performance** : L'application doit être robuste et capable de traiter de gros volumes de données financières sans ralentissement, répondant de manière optimale aux requêtes des utilisateurs.
- **Ergonomie** : L'interface utilisateur (UI) développée doit être simple, intuitive et conviviale, en implémentant les meilleures pratiques de l'expérience utilisateur (UX).
- **Sécurité** : Les données financières étant hautement sensibles, l'accès aux informations n'est possible qu'après une authentification stricte (ex : tokens JWT) et la vérification des privilèges et droits d'accès pour chaque rôle.

- **Rapidité de traitement** : Compte tenu de la complexité des calculs (Worktime overrides, taux journaliers), il est impératif que le temps d'exécution des traitements backend soit le plus court possible.
- **Accessibilité** : Le système doit supporter l'accès simultané de plusieurs utilisateurs (chefs de projets, comptables) de manière fluide et sans conflit de données.
- **Compatibilité** : L'application Web doit être compatible avec les différents systèmes d'exploitation et les navigateurs modernes (Chrome, Firefox, Edge, etc.).

1.3 Conduite et planification du projet

1.3.1 Processus de développement

Le processus de développement constitue un axe déterminant dans la réussite d'un projet, du fait qu'il cadre ses différentes phases et caractérise les principaux traits de sa conduite. Pour cela, le choix d'une méthode de développement adéquate aux particularités et aux exigences du projet doit être élaboré au préalable afin d'obtenir un produit de qualité, répondant aux besoins et attentes des utilisateurs dans des délais et des coûts prévisibles.

La réalisation du module **Invoicing** a été menée au sein d'une équipe de développement de NTT DATA. La gestion du projet a consisté à diviser la tâche globale en adoptant une approche **Agile** basée sur le framework **SCRUM**. Les sous-tâches ont ensuite été assignées aux membres de l'équipe via des outils de suivi. Le calendrier du projet a été minutieusement planifié afin que tous les membres de l'équipe connaissent les délais à respecter.

1.3.2 La méthode SCRUM

Le principe de cette méthode est de concentrer l'équipe sur un ensemble de fonctionnalités à réaliser dans des intervalles de temps fixes, d'une durée de 2 à 4 semaines, appelés **Sprints**.

Chaque Sprint possède un objectif précis défini par le *Product Owner* (Directeur de produit). Au cours de ce cycle, l'équipe de développement conçoit, teste et implémente de nouvelles fonctionnalités (User Stories). À la fin de chaque itération, l'équipe livre des versions intermédiaires et fonctionnelles du projet, permettant ainsi d'avoir un retour continu avant la livraison du produit final. [8]

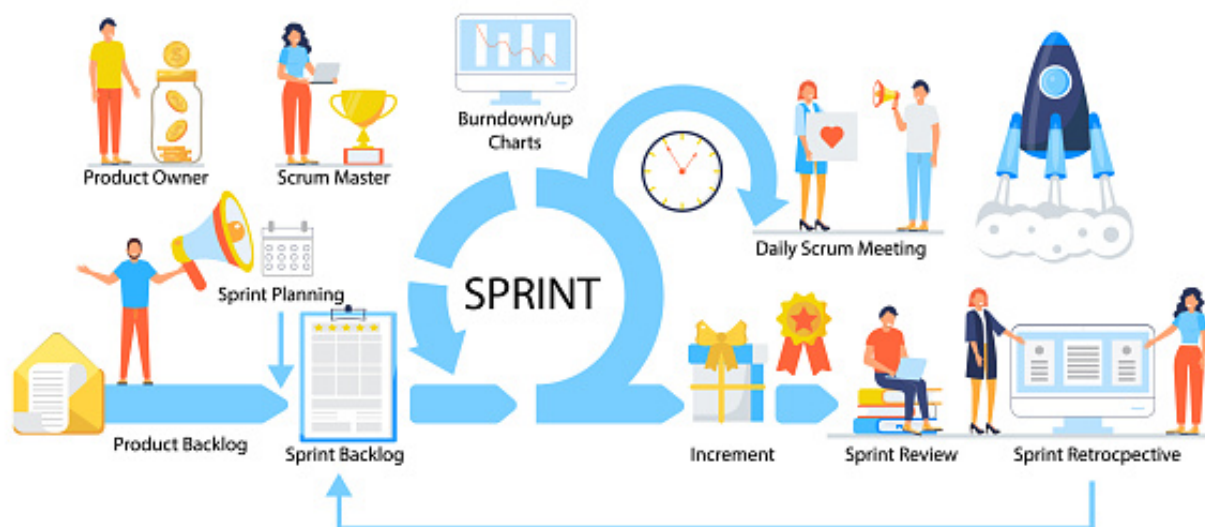


FIGURE 1.11 – La méthode SCRUM

1.3.2.1 Pourquoi SCRUM ?

Le choix de la méthodologie SCRUM pour le développement du module **Invoicing** repose sur plusieurs avantages majeurs :

Avec SCRUM, l'équipe de développement bénéficie d'une participation active dans l'évolution du projet. Elle peut s'exprimer librement et s'impliquer dans la prise de décisions, la planification des sprints et l'organisation du travail, contrairement aux méthodes traditionnelles qui figent les spécifications dès le départ et obligent l'équipe à répondre strictement à un cahier des charges initial.

De plus, le fait de présenter des livraisons régulières (incréments) permet de détecter rapidement les éventuelles défaillances, de signaler les blocages et d'ajuster les fonctionnalités selon les besoins réels. Ainsi, le découpage du projet en Sprints rend le travail moins abstrait et facilite la productivité de l'équipe tout au long du cycle de développement.

1.3.2.2 Les principes fondamentaux de SCRUM

- **Sprints** : Ce sont des itérations de développement de courte durée, généralement de 2 à 4 semaines, au cours desquelles des fonctionnalités spécifiques sont développées, testées et livrées. Chaque sprint aboutit à un incrément de produit potentiellement livrable.
- **Product Backlog (Backlog du produit)** : C'est une liste priorisée et évolutive des fonctionnalités à développer. Le *Product Owner* (Responsable produit) est chargé de sa gestion et de sa mise à jour régulière en fonction des nouveaux besoins et des retours des parties prenantes.
- **Sprint Backlog (Backlog du Sprint)** : Il s'agit de l'ensemble des éléments du Product

Backlog qui ont été sélectionnés pour être réalisés durant le Sprint en cours. C'est le plan de travail de l'équipe de développement.

- **Daily Scrum (Mêlée quotidienne)** : Une courte réunion quotidienne (généralement 15 minutes) permettant à l'équipe de développement de se synchroniser, de faire le point sur l'avancement des tâches et d'identifier les blocages éventuels.
- **Revue et Rétrospective de Sprint** : À la fin de chaque Sprint, l'équipe présente les fonctionnalités réalisées (incrément) aux parties prenantes lors de la Revue. Ensuite, la Rétrospective permet à l'équipe d'analyser son processus de travail afin de l'améliorer pour les Sprints suivants.

1.3.2.3 Équipe du projet

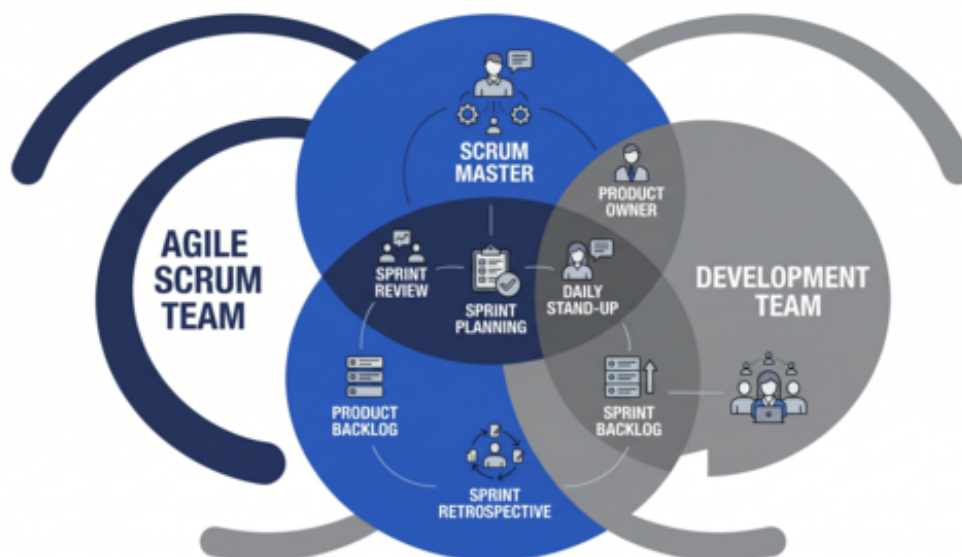


FIGURE 1.12 – L'équipe SCRUM

Pour réaliser notre module d'Invoicing, nous avons intégré une équipe constituée de :

- **Scrum Master (Encadrant professionnel) - M. Achraf Saloumi** : Son rôle est de s'assurer de l'implication de chaque membre, de faciliter le processus Scrum et de nous aider à franchir les différents obstacles que nous pourrions rencontrer.
- **Architecte Backend - M. Ayoub El Youssoufi** : Son rôle est de nous superviser dans la conception architecturale et les tâches de la partie backend.
- **Architecte Frontend - M. El Harraj Abdelatif** : Son rôle est de nous superviser dans les choix techniques et les tâches de la partie frontend.
- **Équipe de développement - Redouane Ed-Dahmany, Soufiane Hajji et Mehdi Farhloumi** : Notre rôle consiste à concevoir, développer et tester les différentes fonctionnalités du projet.

1.3.2.4 Gestion de projet avec JIRA

Comme la méthodologie de travail Scrum que nous utilisons est une approche Agile, l'outil **JIRA** s'est imposé comme la solution idéale pour gérer et suivre l'avancement de notre projet.

Avec JIRA, une plateforme incontournable de gestion de projets, nous pouvons suivre les différentes étapes du cycle de développement, y compris la spécification du backlog et la planification des sprints.



FIGURE 1.13 – Interface de gestion de projet JIRA

Voici comment cet outil est appliqué dans notre contexte :

Spécification du backlog :

- Utilisation des fonctionnalités de gestion des tâches et des User Stories de JIRA pour définir le backlog du projet.
- Classement des fonctionnalités par ordre de priorité selon la valeur métier apportée.

Sprint planning :

- JIRA est utilisé pour visualiser le backlog global et sélectionner les User Stories qui doivent obligatoirement être réalisées pendant le Sprint en cours.
- Le Scrum Master et l'équipe valident les développements nécessaires pour atteindre l'objectif du Sprint.

Estimation des tâches :

- Au début de chaque Sprint, une réunion de planification est organisée avec l'équipe de développement pour estimer la complexité des tâches et établir le Sprint Planning.
- L'équipe de développement estime les tâches à réaliser pendant le sprint en utilisant des points de complexité (Story Points) ou d'effort.
- Les tâches sont ensuite assignées aux différents membres de l'équipe de développement en utilisant les fonctionnalités de gestion et d'attribution de JIRA.

Suivi du sprint :

- Une fois le Sprint lancé, le tableau de bord JIRA (Scrum Board) est utilisé pour suivre visuellement l'avancement des tâches en temps réel.
- Les développeurs peuvent mettre à jour l'état de leurs tâches en les déplaçant de manière fluide à travers les différentes colonnes (À faire, En cours, En test, Terminé).



Dans l’optique d’optimiser le cycle de vie du développement et d’assurer une implémentation robuste du module **Invoicing**, une répartition judicieuse des charges de travail a été instaurée au sein de notre équipe technique. En parfaite adéquation avec notre démarche Agile, l’attribution des périmètres d’intervention — qu’ils soient fonctionnels ou architecturaux — a été pensée de manière à maximiser la synergie du groupe et à garantir une couverture exhaustive des exigences du projet. Le tableau ci-dessous détaille cette affectation des rôles et des missions pour chaque membre :

Membre de l'équipe	Modules affectés	Détails des tâches
Redouane Ed-Dahmany (Moi-même)	Backend : Worktime, Billing, Export PDF Frontend : UI Admin, Settings, Massive Edit	• Développement backend de la gestion du Worktime. • Implémentation de la logique de facturation (Billing). • Développement de la génération des factures en PDF. • Création des interfaces d'administration. • Conception de la vue de modification massive. • Intégration des APIs REST côté Angular.
Soufiane Hajji	Backend : Analyse UML, BDD, Service CRUD, Group Mgmt, Ajustements, Sécurité	• Modélisation UML et conception de la base de données. • Développement des services CRUD et des groupes. • Implémentation de la logique des ajustements (Overrides). • Configuration de la sécurité (Spring Security & JWT). • Écriture des tests unitaires backend.
Mehdi Farhloumi	Frontend : Generate, Confirm & Close Service, UI/UX, Routage	• Frontend de génération et clôture des services. • Consommation des APIs REST via les services Angular. • Gestion et validation des formulaires (Reactive Forms). • Mise en place du routage et protection (Auth Guards). • Amélioration de l'expérience utilisateur (UI/UX).

TABLE 1.2 – Détail approfondi des modules et tâches par membre de l'équipe

Grâce à cette répartition des tâches, chaque membre de l'équipe a pu se concentrer sur des aspects spécifiques du projet, en travaillant de manière collaborative pour assurer la réalisation efficace de chaque module. L'implication de l'équipe de développement dans ces domaines clés garantit un développement robuste et une meilleure expérience utilisateur tout au long du processus de gestion de la facturation et du suivi des temps de travail.

1.3.2.6 Organisation et communication (Microsoft Teams)

Dans le cadre de l'organisation de notre projet et pour maintenir une synchronisation parfaite, nous avons adopté **Microsoft Teams** comme outil principal pour nos réunions quotidiennes et notre communication continue.

Microsoft Teams est une plateforme de communication et de collaboration en ligne qui offre diverses fonctionnalités pour faciliter les échanges, la coordination et le suivi de l'avancement au sein de l'équipe de travail.



FIGURE 1.15 – Plateforme de communication Microsoft Teams

Les réunions quotidiennes (*Daily Scrum*) sont un élément essentiel de la gestion de projet Agile, car elles permettent de mettre à jour l'équipe sur l'avancement des tâches, d'identifier les problèmes éventuels et de prendre des décisions importantes. En utilisant Microsoft Teams, nous avons pu organiser facilement ces réunions virtuelles, garantissant ainsi une fluidité même lors des séances de télétravail.

Voici quelques-unes des fonctionnalités de Microsoft Teams que nous avons exploitées pour optimiser notre organisation :

- **Appels audio et vidéo** : Nous avons pu effectuer des appels audio ou vidéo avec les membres de l'équipe, ce qui a permis une communication en temps réel et une meilleure compréhension des informations partagées lors des Daily Scrums.
- **Partage d'écran** : Nous avons utilisé la fonction de partage d'écran pour présenter des informations, des rapports d'erreurs, des revues de code ou des démonstrations lors de nos réunions. Cela a favorisé une résolution plus rapide des problèmes (Pair programming) et une collaboration plus efficace.
- **Chat en ligne** : En dehors des réunions, nous avons utilisé le chat en ligne pour les discussions rapides, les échanges d'informations techniques et les clarifications. Cela a facilité la communication instantanée et a permis de résoudre rapidement les points bloquants.

- **Gestion des documents :** Teams offre une intégration native avec l'écosystème Microsoft Office, ce qui nous a permis de partager et de collaborer sur des documents, des diagrammes UML et des spécifications techniques directement dans l'environnement de l'outil. Cela a simplifié le processus de travail et a évité les problèmes liés au contrôle des versions des fichiers.

En utilisant Microsoft Teams pour nos réunions quotidiennes, nous avons pu maintenir une communication régulière et efficace au sein de l'équipe, même à distance. Cela nous a permis de rester alignés sur les objectifs du sprint, de résoudre rapidement les problèmes bloquants et de prendre des décisions informées. De plus, l'utilisation de cette plateforme a fortement favorisé la centralisation des informations, des spécifications techniques et des documents, ce qui a simplifié la collaboration et l'organisation globale des tâches liées au module Invoicing.

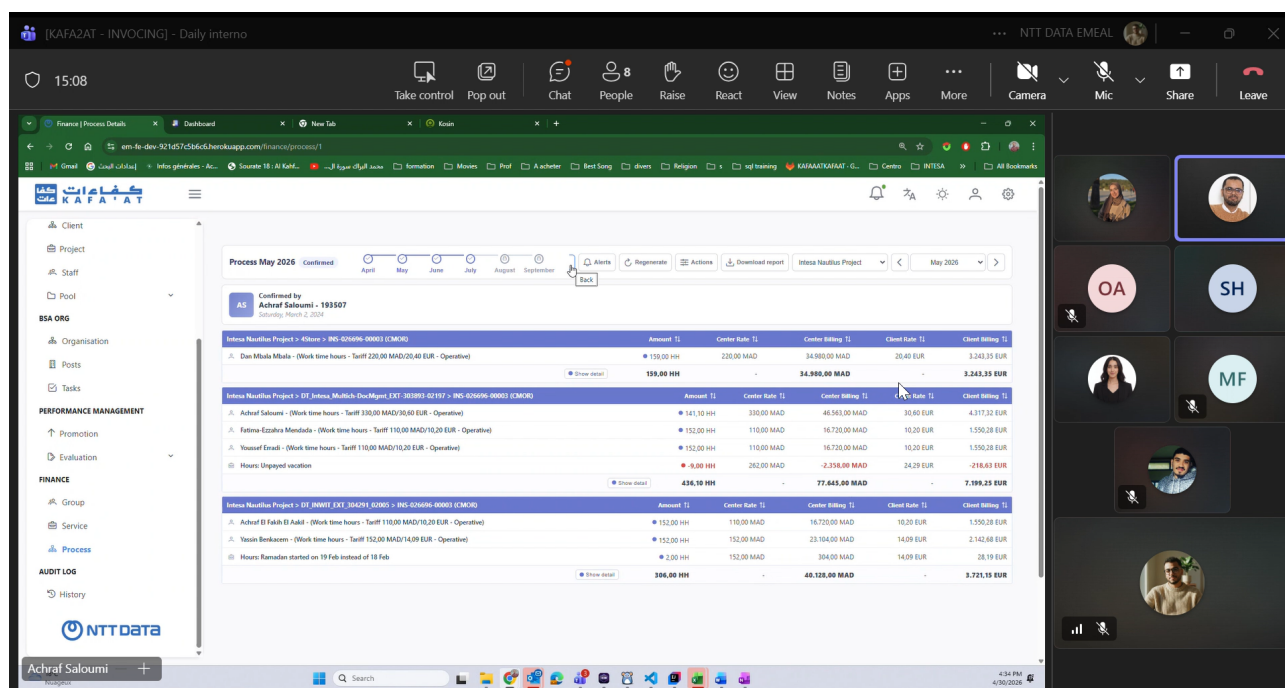


FIGURE 1.16 – Collaboration de l'équipe sur Microsoft Teams

1.3.3 Planification temporelle

La planification du projet est le processus de détermination des objectifs, des activités, des ressources et des délais nécessaires pour atteindre un résultat spécifique dans un projet. Cela implique l'identification des tâches clés, l'estimation des ressources requises, l'établissement d'un calendrier et l'allocation des responsabilités. La planification du projet permet de structurer et d'organiser les différentes étapes du projet, de définir les priorités, d'évaluer les contraintes et de minimiser les risques potentiels. Une planification solide aide à coordonner efficacement les efforts de l'équipe, à optimiser l'utilisation des ressources et à assurer la livraison réussie du projet dans les délais impartis.

1.3.3.1 Diagramme de GANTT

Le diagramme de Gantt est un outil de planification et de gestion de projet qui représente visuellement les différentes tâches d'un projet, leur séquence et leur durée. Il est constitué d'une ligne horizontale représentant la période totale du projet, et des barres horizontales qui indiquent le début et la fin de chaque tâche. Les tâches sont organisées en fonction de leur dépendance et de leur ordre logique. [10]

Le diagramme ci-dessous a été réalisé à l'aide de l'outil de modélisation **Mermaid** :

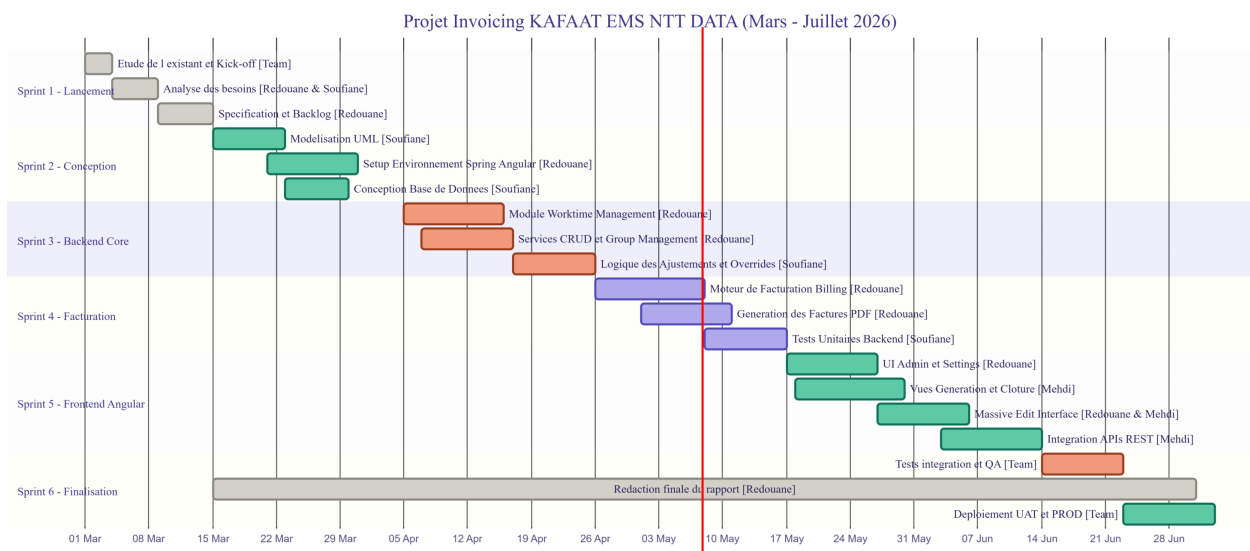


FIGURE 1.17 – Diagramme de GANTT du projet Invoicing

1.3.3.2 Découpage en Sprints

Le tableau ci-dessous représente la description de chaque sprint, ce qui offre une vision globale de notre travail pendant la durée du stage :

Activité (Sprint)	Durée	Date de début	Date de fin
Sprint 1 : Lancement - Étude de l'existant et Kick-off - Analyse des besoins - Spécification et Backlog	14 jours	01/03/2026	14/03/2026
Sprint 2 : Conception - Modélisation UML - Setup Environnement Spring/Angular - Conception Base de Données	16 jours	15/03/2026	30/03/2026
Sprint 3 : Backend Core - Module Worktime Management - Services CRUD et Group Management - Logique des Ajustements et Overrides	21 jours	05/04/2026	25/04/2026
Sprint 4 : Facturation - Moteur de Facturation (Billing) - Génération des Factures PDF - Tests Unitaires Backend	21 jours	26/04/2026	16/05/2026
Sprint 5 : Frontend Angular - UI Admin et Settings - Vues Génération et Clôture - Massive Edit Interface - Intégration APIs REST	28 jours	17/05/2026	13/06/2026
Sprint 6 : Finalisation - Tests d'intégration et QA - Rédaction finale du rapport - Déploiement UAT et PROD	19 jours	14/06/2026	02/07/2026

TABLE 1.3 – Description globale et temporelle des Sprints

Conclusion

Durant ce premier chapitre, nous avons réussi à fournir une vue d'ensemble du projet en le situant dans son contexte général. Nous avons introduit l'organisme d'accueil, NTT DATA, et nous avons décrit de manière détaillée la problématique à résoudre, ainsi que les objectifs à atteindre dans le cadre du développement du module **Invoicing** pour la plateforme **KAFAT EMS**.

De plus, nous avons abordé le choix de la méthodologie de travail adoptée (Agile SCRUM) pour assurer la réussite et la flexibilité de réalisation. Nous avons également souligné l'importance de la communication et de la planification pour la gestion efficace des différentes étapes et itérations du projet.

En conclusion, ce premier chapitre nous a permis de poser les fondations du projet en définissant son cadre, ses objectifs et sa méthodologie. Il nous a fourni une compréhension globale du besoin et nous a préparés pour la suite, où nous aborderons les détails spécifiques de l'analyse fonctionnelle et de la conception de notre solution.

Chapitre 2

Analyse et Conception

Introduction

Dans ce chapitre d'analyse et de conception, nous utiliserons des outils de modélisation tels que les diagrammes UML pour approfondir notre compréhension du projet et détailler l'architecture du module **Invoicing** au sein de la plateforme **KAFA'AT EMS**.

Nous commencerons par décrire les besoins fonctionnels à travers les cas d'utilisation, puis nous concevrons la structure du système à l'aide de diagrammes de classes. Ensuite, nous utiliserons les diagrammes de séquence pour représenter les interactions chronologiques entre les différents acteurs et notre backend Spring Boot. Enfin, les diagrammes d'activité nous aideront à visualiser les flux de travail et les processus de facturation. Ces outils de modélisation nous guideront dans la conception détaillée et la réalisation technique du système.

2.1 Langage UML

2.1.1 Définition

Le langage UML (*Unified Modeling Language*) est un langage de modélisation graphique utilisé pour représenter, spécifier, visualiser et documenter les différents aspects d'un système logiciel. Il fournit un ensemble de notations graphiques standardisées qui permettent aux développeurs, aux architectes et aux analystes de communiquer et de comprendre les concepts, les structures, les comportements et les interactions d'un système.

Le langage UML comprend plusieurs types de diagrammes, tels que les diagrammes de cas d'utilisation, les diagrammes de classes, les diagrammes de séquence, les diagrammes d'activité, et les diagrammes de déploiement. Chaque type de diagramme a une utilisation spécifique et permet de représenter différents aspects du système.

En utilisant UML, les professionnels de l'informatique peuvent modéliser et concevoir des systèmes logiciels de manière claire et précise. Les diagrammes UML servent de documentation visuelle pour décrire l'architecture, les fonctionnalités, les interactions et les dépendances du système, ce qui facilite la compréhension, la communication et la collaboration entre les différentes parties prenantes de notre projet. [11]



FIGURE 2.1 – Logo officiel du langage UML

2.1.2 Intérêts de la modélisation UML

Le choix du langage UML pour la phase de conception du module **Invoicing** ne s'est pas fait au hasard. Il s'appuie sur plusieurs atouts majeurs qui répondent directement aux exigences de notre projet :

- **Communication** : UML fournit un langage commun et standardisé. Il a grandement facilité la collaboration et les échanges techniques entre les membres de l'équipe (développeurs et encadrants au sein de NTT DATA) en offrant une vision partagée et claire de l'architecture du système.
- **Modélisation visuelle** : UML utilise des diagrammes graphiques pour représenter visuellement la structure et le comportement du système. Cela nous a permis de traduire les règles métiers complexes de la facturation en concepts clairs et intuitifs.
- **Abstraction et encapsulation** : UML favorise une approche orientée objet. Ceci correspond parfaitement aux technologies que nous avons adoptées pour ce projet (**Java Spring Boot** pour le backend et **Angular** pour le frontend), facilitant ainsi le passage de la conception au code tout en garantissant une architecture modulaire.
- **Documentation** : Les diagrammes UML servent de documentation visuelle de référence pour décrire l'architecture, les contraintes et les flux du système. Ils constituent une base solide pour la maintenance et les futures évolutions de la plateforme KAFA'AT EMS.
- **Support d'outils** : De nombreux outils de modélisation prennent en charge UML (comme Draw.io, PlantUML ou StarUML), ce qui simplifie la création, la modification et l'intégration de ces modèles directement dans notre documentation technique.

En résumé, l'adoption d'UML pour la conception de ce projet nous permet de maîtriser la complexité du module de facturation, d'encapsuler la logique métier et de fournir une documentation claire, tout en bénéficiant d'une transition fluide vers l'implémentation logicielle. [11]

2.2 Modélisation du système

2.2.1 Diagramme de cas d'utilisation

Un diagramme de cas d'utilisation est un outil de modélisation dans le langage UML qui représente les interactions entre les acteurs (utilisateurs, systèmes externes ou administrateurs) et un système logiciel. Il permet de capturer les exigences fonctionnelles en décrivant *ce que* le système doit faire, sans pour autant spécifier *comment* il va le faire.

Il permet de visualiser les fonctionnalités du système du point de vue des utilisateurs, en identifiant les différentes actions qu'ils peuvent effectuer et les résultats attendus. Le diagramme de cas d'utilisation est utile pour décrire les exigences fonctionnelles du module **Invoicing**, définir son périmètre et faciliter la communication entre les parties prenantes. Il aide à comprendre comment les utilisateurs interagissent avec la plateforme KAFA'AT EMS et à concevoir des

fonctionnalités centrées sur leurs besoins. [11]

2.2.1.1 Identification des acteurs

Dans le cadre de notre module, nous avons identifié deux acteurs principaux : l'administrateur (*Admin*) et l'utilisateur standard (*User* / Employé). L'administrateur possède toutes les fonctionnalités de l'utilisateur, ainsi que des privilèges supplémentaires liés au paramétrage et à la facturation.

L'utilisateur standard peut effectuer diverses actions telles que la consultation de son temps de travail (*Worktime*), la visualisation de ses factures générées, et le suivi de ses données. Ces fonctionnalités permettent à l'utilisateur d'interagir avec le système pour accomplir des tâches spécifiques liées à son profil au sein de NTT DATA.

De son côté, seul l'administrateur a le privilège de configurer les paramètres de facturation, de gérer les groupes d'utilisateurs, d'appliquer des ajustements (*overrides*) et de déclencher le moteur de génération des factures PDF. Ces fonctionnalités critiques sont strictement réservées à l'administrateur, qui possède des droits étendus pour superviser l'ensemble du processus de facturation.

La relation de généralisation entre l'administrateur et l'utilisateur indique que l'administrateur est une catégorie spéciale d'utilisateur qui bénéficie de fonctionnalités et de droits supplémentaires. Cette hiérarchie permet une gestion efficace des autorisations, des rôles et des permissions au sein de notre backend Spring Boot.

En plus de ces deux acteurs humains, nous avons intégré un **Serveur SMTP (Système Externe)** qui interagit automatiquement avec notre application pour l'envoi des notifications et des factures par courriel aux employés de manière asynchrone.

2.2.1.2 Diagramme de cas d'utilisation général de l'application

L'utilisateur (*User* / Employé) peut consulter son tableau de bord, visualiser le suivi de ses temps de travail (*Worktime*), et consulter ou télécharger ses propres factures. Ces actions nécessitent obligatoirement que l'utilisateur s'authentifie au préalable pour pouvoir accéder aux fonctionnalités du système en toute sécurité.

L'administrateur (*Admin*), en plus des fonctionnalités de base disponibles pour l'utilisateur, dispose de privilèges supplémentaires liés au cœur de métier de l'application. Il a la capacité de configurer les paramètres du moteur de facturation (*Billing*), de gérer les groupes d'employés, d'appliquer des ajustements ou des exceptions (*Overrides*) sur les heures enregistrées, et de déclencher la génération et la clôture des factures au format PDF.

En outre, le système fait appel à un acteur externe, le **Serveur SMTP (Messagerie)**, qui est sollicité automatiquement pour notifier les employés par courriel lors de la génération de leurs factures ou en cas de modifications de leurs pointages.

Il existe une relation de généralisation entre l'administrateur et l'utilisateur, ce qui signifie conceptuellement que l'administrateur est un type spécial d'utilisateur. Il hérite de toutes les actions de l'utilisateur standard tout en bénéficiant de fonctionnalités et de droits d'accès plus étendus pour la gestion de la facturation.

La figure ci-dessous montre le diagramme général des cas d'utilisation du projet :

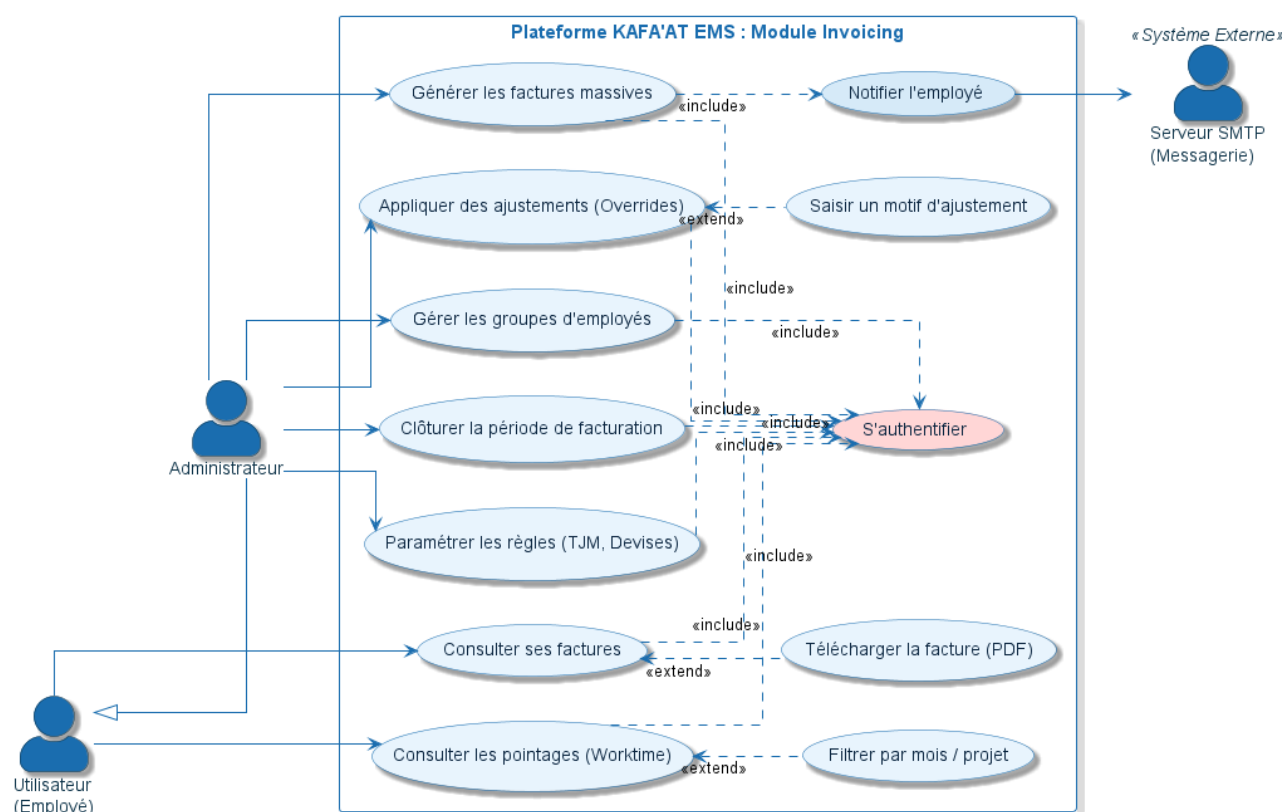


FIGURE 2.2 – Diagramme de cas d'utilisation général du module Invoicing

2.2.1.3 Diagramme de cas d'utilisation de gestion du temps de travail

L'utilisateur (*User* / Employé) peut consulter ses propres pointages, soumettre sa feuille de temps, filtrer les pointages existants en fonction de certains critères et exporter le relevé. Pour accéder à ces fonctionnalités, l'utilisateur doit s'authentifier au préalable.

L'administrateur (*Admin*), en plus des fonctionnalités disponibles pour l'utilisateur, possède des privilèges supplémentaires. Il peut consulter les pointages globaux de tous les employés, gérer les jours fériés et les absences, et valider la période mensuelle. De plus, l'administrateur a la possibilité d'appliquer des ajustements (*Overrides*) ou de refuser des pointages. Ces deux actions spécifiques exigent la saisie d'un motif justificatif et déclenchent l'envoi d'une alerte à l'employé via le système externe.

La relation de généralisation entre l'administrateur et l'utilisateur indique que l'administrateur possède toutes les fonctionnalités de l'utilisateur.

La figure ci-dessous montre le diagramme de cas d'utilisation de gestion du temps de travail :

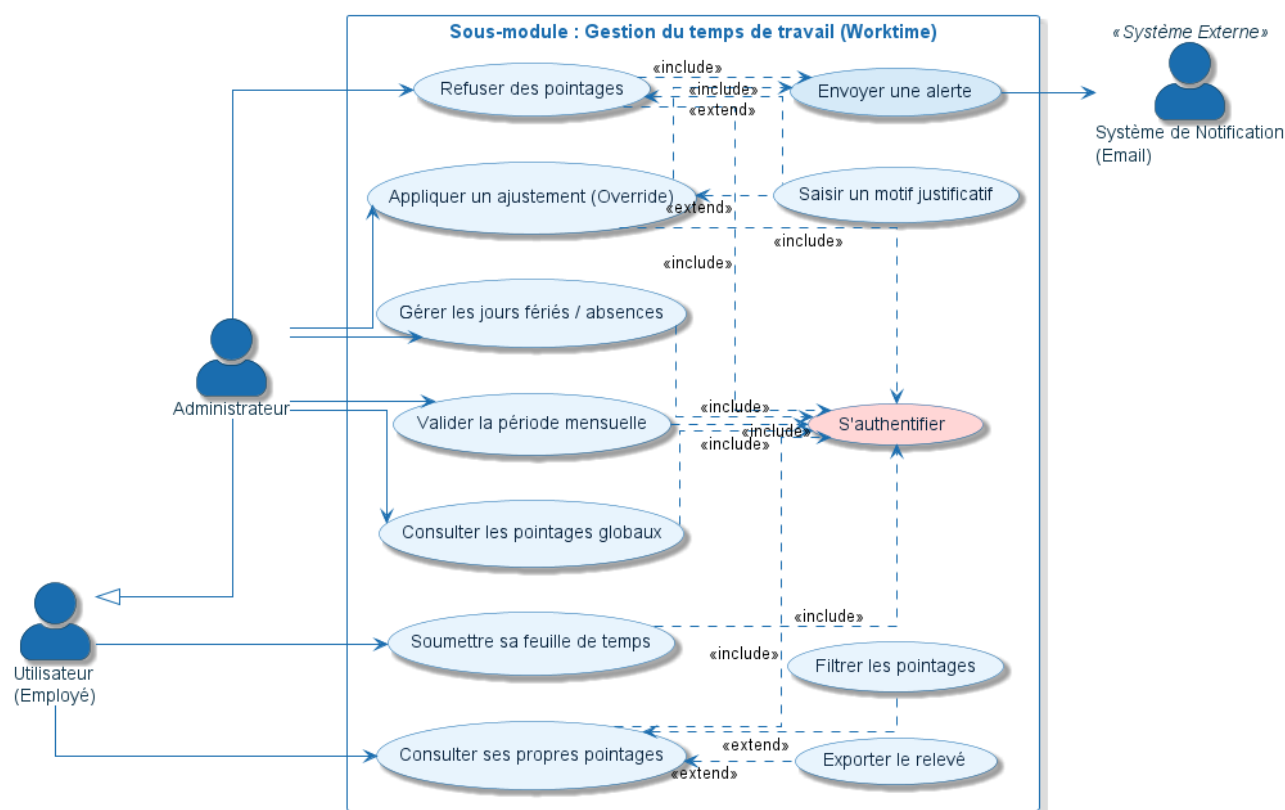


FIGURE 2.3 – Diagramme de cas d'utilisation de gestion du temps de travail

2.2.1.4 Diagramme de cas d'utilisation de gestion de la facturation

L'utilisateur (*User* / Employé) peut consulter l'historique de ses factures et a également la possibilité de télécharger ses factures au format PDF.

L'administrateur (*Admin*) dispose de fonctionnalités supplémentaires pour gérer le cœur du processus de facturation. Il peut paramétrer les règles de facturation telles que les TJM et les taxes, modifier massivement les factures, lancer la génération massive des factures au format PDF et clôturer la période de facturation. De plus, la génération des factures déclenche automatiquement l'envoi de la facture par email à l'employé via un système de notification externe.

Toutes ces actions nécessitent une authentification préalable pour accéder aux fonctionnalités de gestion de la facturation. La relation de généralisation entre l'administrateur et l'utilisateur indique que l'administrateur possède toutes les fonctionnalités de l'utilisateur.

La figure ci-dessous montre le diagramme de cas d'utilisation de gestion de la facturation :

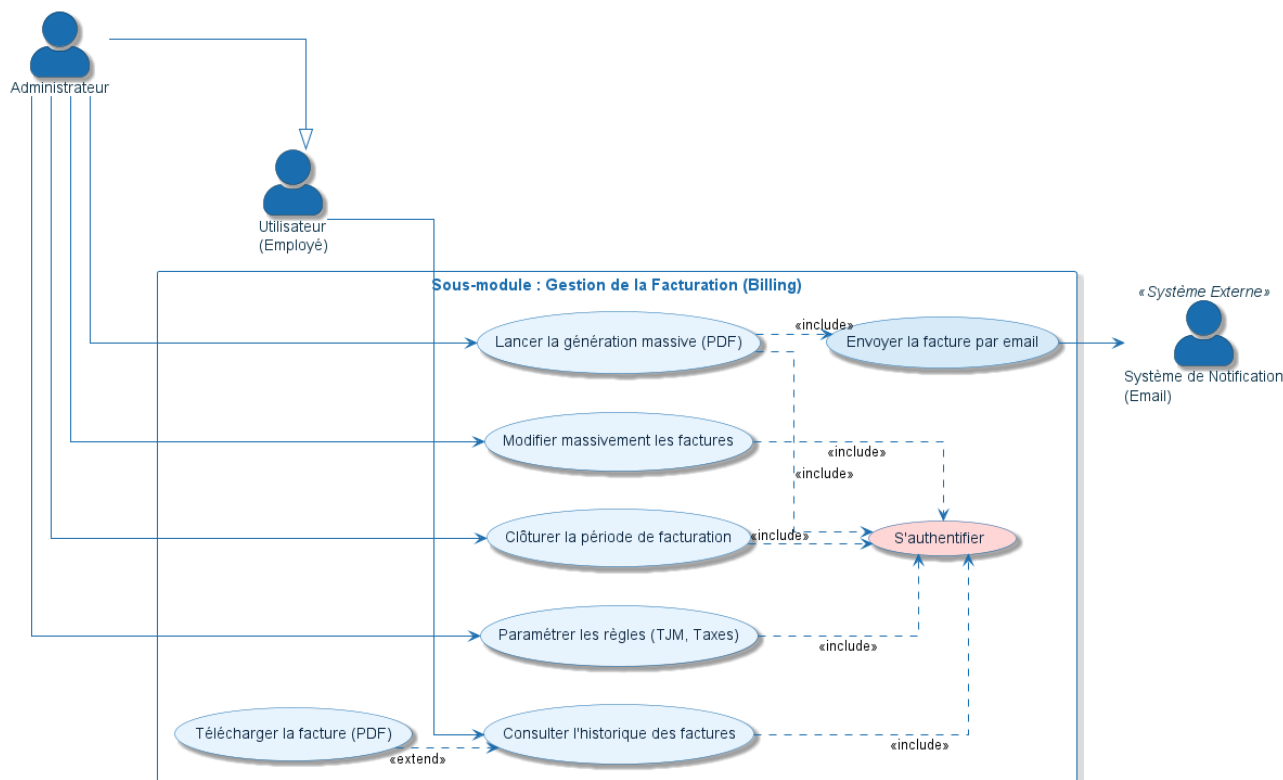


FIGURE 2.4 – Diagramme de cas d'utilisation de gestion de la facturation

2.2.1.5 Diagramme de cas d'utilisation de gestion des groupes et tarification

L'utilisateur (*User / Employé*) dispose d'un accès restreint dans cette section. Il a uniquement la possibilité de consulter son propre profil de facturation pour visualiser les modalités qui lui sont appliquées.

L'administrateur (*Admin*) dispose de privilèges supplémentaires, en tant que généralisation de l'utilisateur. Il peut créer un nouveau groupe de facturation (avec la possibilité de définir un plafond d'heures), consulter la liste complète des groupes, modifier un groupe existant, ou encore supprimer un groupe du système. De plus, il a la capacité d'affecter des employés aux différents groupes et de définir le Taux Journalier Moyen (TJM) en spécifiant, si nécessaire, une devise (MAD, EUR, USD). Enfin, l'administrateur peut filtrer la liste des groupes et l'exporter au format Excel.

Toutes ces actions nécessitent une authentification préalable pour accéder aux fonctionnalités de gestion des groupes et de la tarification.

La figure ci-dessous montre le diagramme de cas d'utilisation de gestion des groupes et de la tarification :

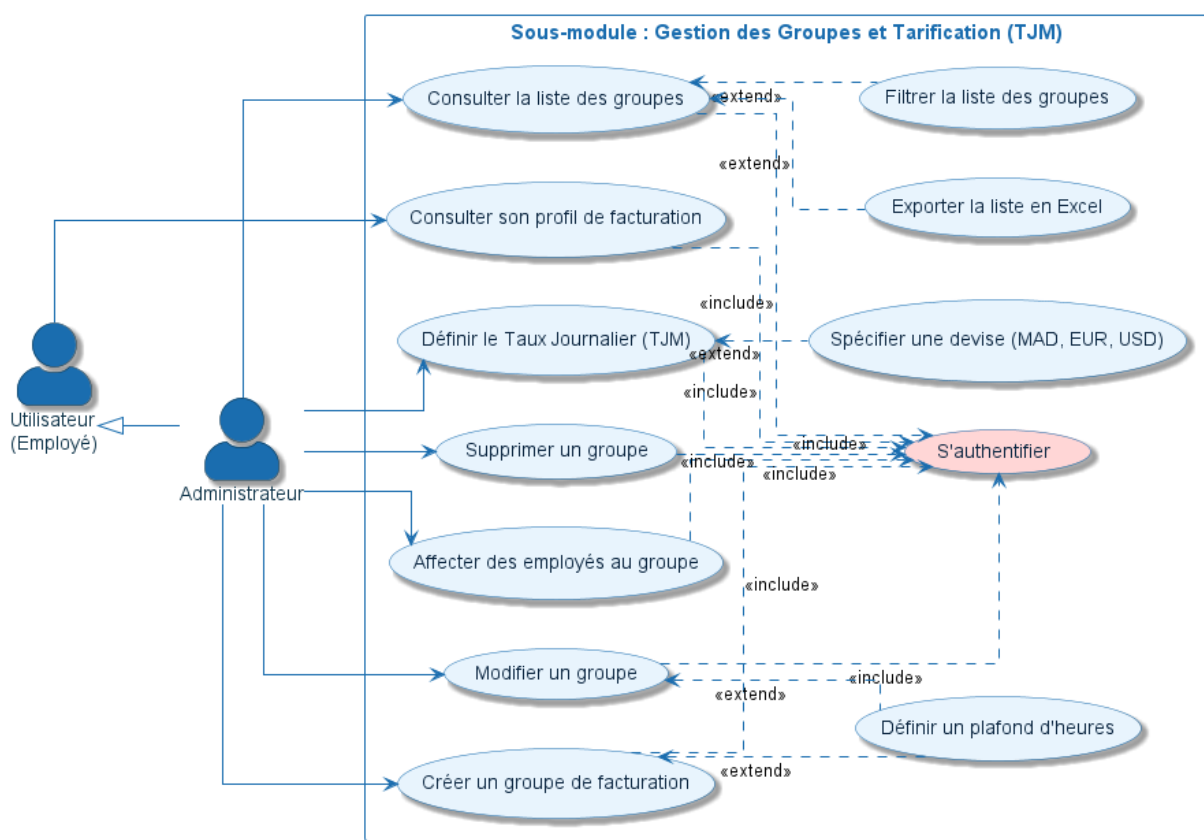


FIGURE 2.5 – Diagramme de cas d'utilisation de gestion des groupes et tarification

2.2.1.6 Diagramme de cas d'utilisation de gestion des utilisateurs

L'utilisateur (*User / Employé*) a la possibilité de consulter son propre profil pour visualiser les informations qui y sont associées. Il peut également mettre à jour ses coordonnées personnelles et changer son mot de passe pour des raisons de sécurité.

L'administrateur (*Admin*) a des fonctionnalités supplémentaires. Il peut consulter la liste complète des employés enregistrés dans le système, créer de nouveaux comptes, et modifier les comptes existants. Lors de la création ou de la modification d'un compte, l'administrateur a la possibilité d'attribuer des rôles spécifiques (privileges) grâce à un système d'extensions. De plus, il peut réinitialiser le mot de passe d'un utilisateur en cas de besoin. Pour des raisons de traçabilité et d'intégrité des données financières, l'administrateur ne supprime pas définitivement les utilisateurs, mais peut désactiver ou suspendre un compte.

Toutes les actions décrites ci-dessus nécessitent une authentification préalable pour accéder aux fonctionnalités de gestion des utilisateurs. La relation de généralisation entre l'administrateur et l'utilisateur indique que l'administrateur est un type spécial d'utilisateur ayant des privilèges étendus dans le système.

Sous-module : Gestion des Utilisateurs et Accès

Acteurs :

- Administrateur
- Utilisateur (Employé)

Cas d'utilisation :

- Modifier un compte existant
- Créer un nouveau compte
- Consulter la liste des employés
- Désactiver / Suspendre un compte
- Changer son mot de passe
- Consulter son profil
- Mettre à jour ses coordonnées
- Réinitialiser le mot de passe
- Attribuer des rôles (Privilèges)
- S'authentifier

Relations :

- L'Administrateur utilise : Modifier un compte existant, Créer un nouveau compte, Consulter la liste des employés, Désactiver / Suspendre un compte.
- L'Utilisateur utilise : Changer son mot de passe, Consulter son profil, Mettre à jour ses coordonnées.
- «include» : Consulter la liste des employés → S'authentifier; Désactiver / Suspendre un compte → S'authentifier; Changer son mot de passe → S'authentifier; Consulter son profil → S'authentifier; Mettre à jour ses coordonnées → S'authentifier.
- «extend» : Réinitialiser le mot de passe → Modifier un compte existant; Attribuer des rôles (Privilèges) → Créer un nouveau compte.

FIGURE 2.6 – Diagramme de cas d'utilisation de gestion des utilisateurs et des accès

Un diagramme de classes est une représentation graphique des classes, des relations entre les classes et des attributs et méthodes associés à chaque classe. Il est utilisé dans le langage de modélisation unifié (UML) pour visualiser la structure statique d'un système logiciel. Le diagramme de classes présente les différentes classes du système et leurs associations, y compris les relations d'héritage, d'agrégation, d'association et de dépendance entre les classes. Il indique également les attributs (variables) et les méthodes (fonctions) définies dans chaque classe. Ce diagramme permet de comprendre la structure du système, les entités impliquées, leurs caractéristiques et leurs interactions. Il facilite la communication entre les développeurs et les concepteurs en fournissant une vue claire et concise de l'architecture logicielle. [11]

Pour répondre aux exigences complexes de la plateforme KAFA'AT EMS (Module Invoicing), notre modélisation s'articule autour de quatre domaines principaux, mis en évidence par un code couleur spécifique :

- **Utilisateurs et Sécurité (Bleu foncé)** : Les classes **Utilisateur** et **Administrateur** gèrent l'authentification et les profils. Leurs habilitations sont définies par l'énumération **Role**.
- **Clients et Projets (Bleu moyen)** : Le système lie le travail à un **Projet** spécifique, lui-même commandé par un **Client** (l'entreprise qui sera facturée).
- **Gestion du Worktime (Bleu clair)** : La classe **Pointage** enregistre les heures déclarées. Son type (Normale, Supp, Férié) est défini par **TypePointage**. Toute modification effectuée par un administrateur est historisée dans la classe **Ajustement** pour garantir la traçabilité.
- **Moteur de Facturation (Cyan)** : La classe **GroupeFacturation** centralise les règles tarifaires (TJM, Devise). Lors de la clôture, le système génère une **Facture** (liée au **Client** et à l'**Utilisateur**) qui est composée de plusieurs **LigneFacture** détaillant les sous-totaux.

La figure ci-dessous montre le diagramme de classes pour notre projet :

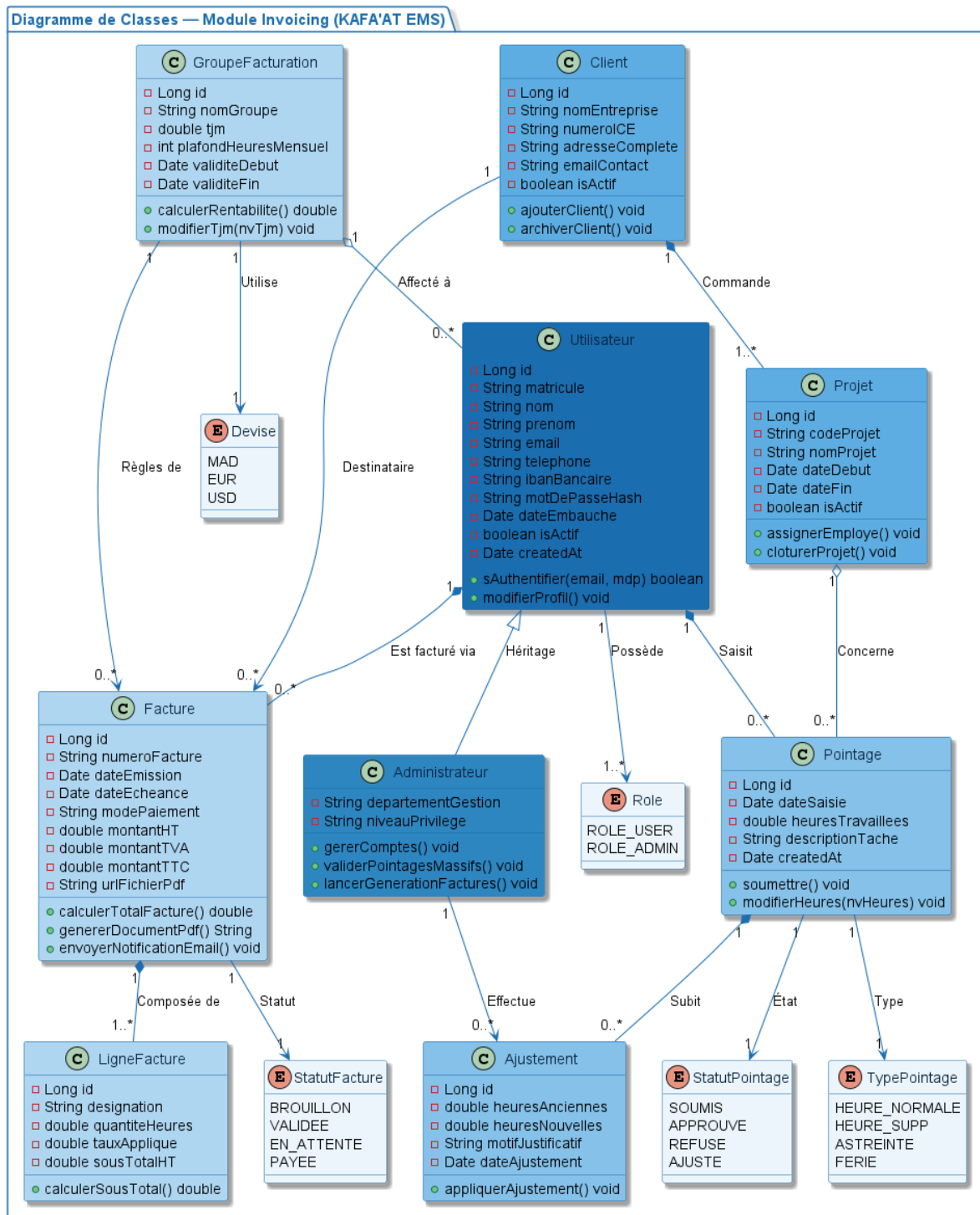


FIGURE 2.7 – Diagramme de classes du module Invoicing et Worktime

2.2.3 Diagramme de séquence

Un diagramme de séquence est un type de diagramme utilisé dans le langage de modélisation unifié (UML) pour représenter la séquence des interactions entre les objets d'un système. Il met l'accent sur la chronologie des messages échangés entre les objets et les actions qu'ils effectuent en réponse à ces messages.

Le diagramme de séquence montre l'ordre dans lequel les messages sont envoyés, ainsi que les objets qui envoient et reçoivent ces messages via des "lignes de vie" verticales. Ce diagramme permet de modéliser le comportement dynamique d'un système en montrant comment les composants (Frontend, Backend, Base de données) interagissent pour accomplir un processus métier spécifique. [11]

2.2.3.1 Diagramme de séquence « S'authentifier »

L'authentification est une étape cruciale pour sécuriser l'accès à la plateforme KAFA'AT EMS. Notre processus s'appuie sur les standards modernes de sécurité de l'architecture distribuée, notamment **Spring Security** et les jetons **JWT (JSON Web Token)**.

La figure ci-dessous illustre cette séquence d'interactions :

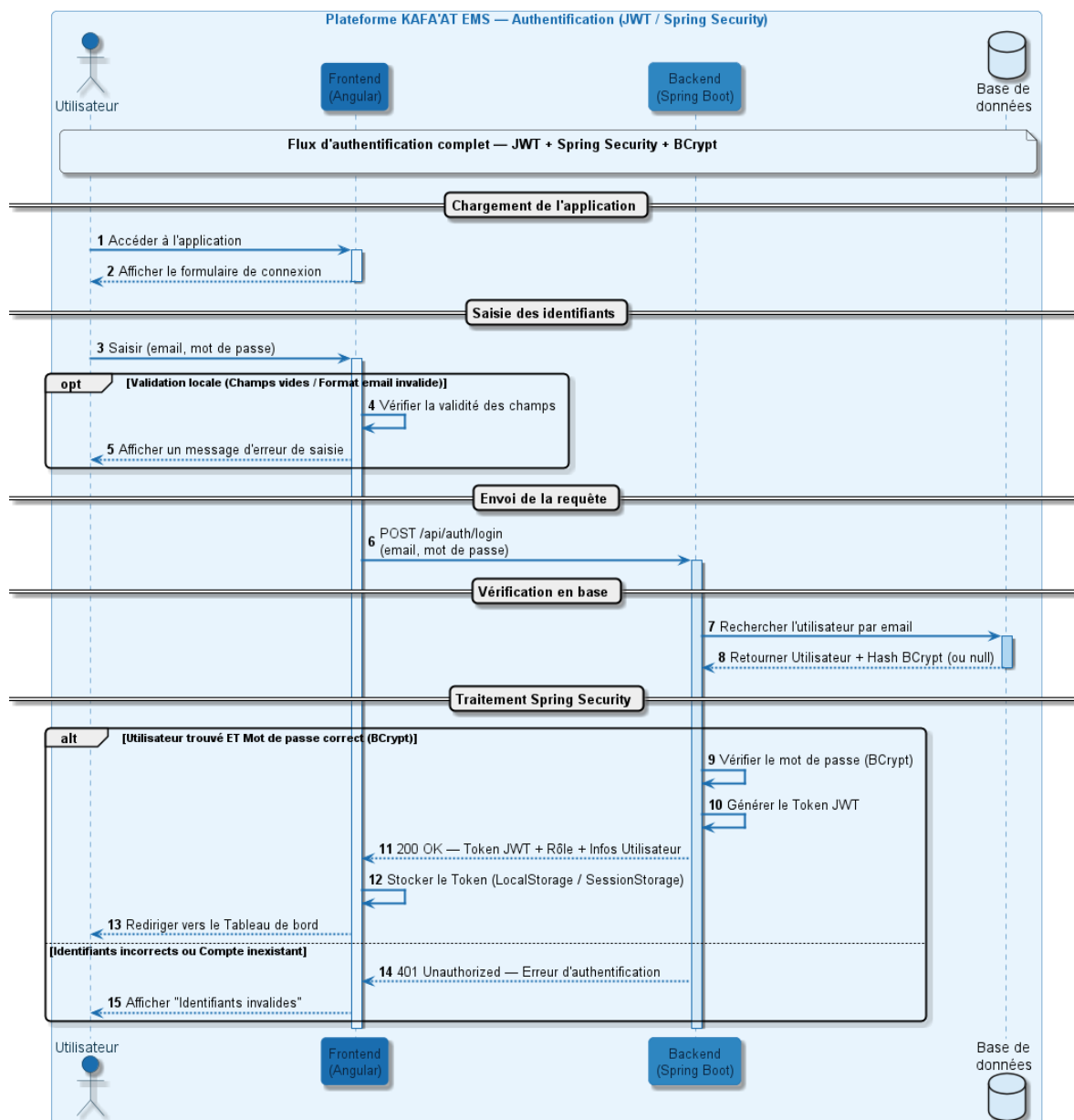


FIGURE 2.8 – Diagramme de séquence d'authentification (JWT & Spring Security)

2.2.3.2 Diagramme de séquence « Soumettre un pointage »

L'enregistrement du temps de travail (Worktime) est l'une des fonctionnalités centrales pour l'employé. Ce processus n'est pas une simple insertion de données ; il implique une validation métier rigoureuse au niveau du **Backend (Spring Boot)** pour vérifier que les règles de l'entreprise sont respectées (par exemple, s'assurer que le cumul d'heures saisies pour une même journée ne dépasse pas le plafond autorisé).

La figure ci-dessous détaille les interactions entre le Frontend (Angular), le Backend et la base de données lors de la soumission d'un pointage :

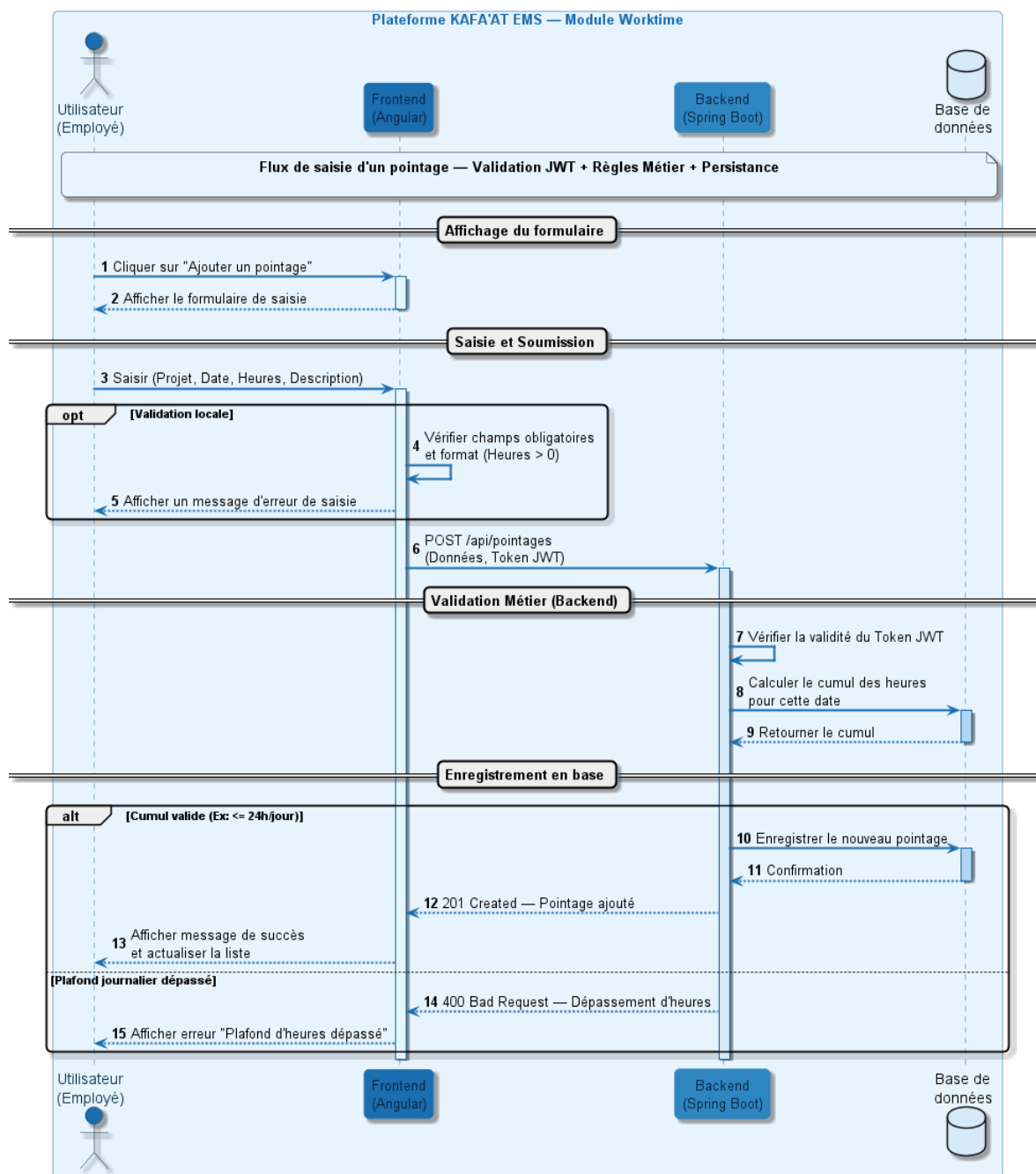


FIGURE 2.9 – Diagramme de séquence de la soumission d'un pointage

2.2.3.3 Diagramme de séquence « Génération massive des factures »

Ce diagramme illustre le cœur du processus métier de notre application, une tâche exclusivement réservée à l'administrateur. La génération massive des factures est une opération complexe qui fait intervenir de multiples composants du système.

Lors du déclenchement, le **Backend (Spring Boot)** vérifie les habilitations, puis interroge la base de données pour récupérer les pointages validés et les taux tarifaires (TJM) associés à chaque groupe d'employés. Le système effectue ensuite les calculs financiers (montants HT, taxes), génère les documents au format **PDF**, et les sauvegarde en base de données. Enfin, de manière asynchrone pour ne pas bloquer l'interface, le backend communique avec un **Serveur SMTP externe** pour envoyer automatiquement les factures par courriel aux collaborateurs concernés.

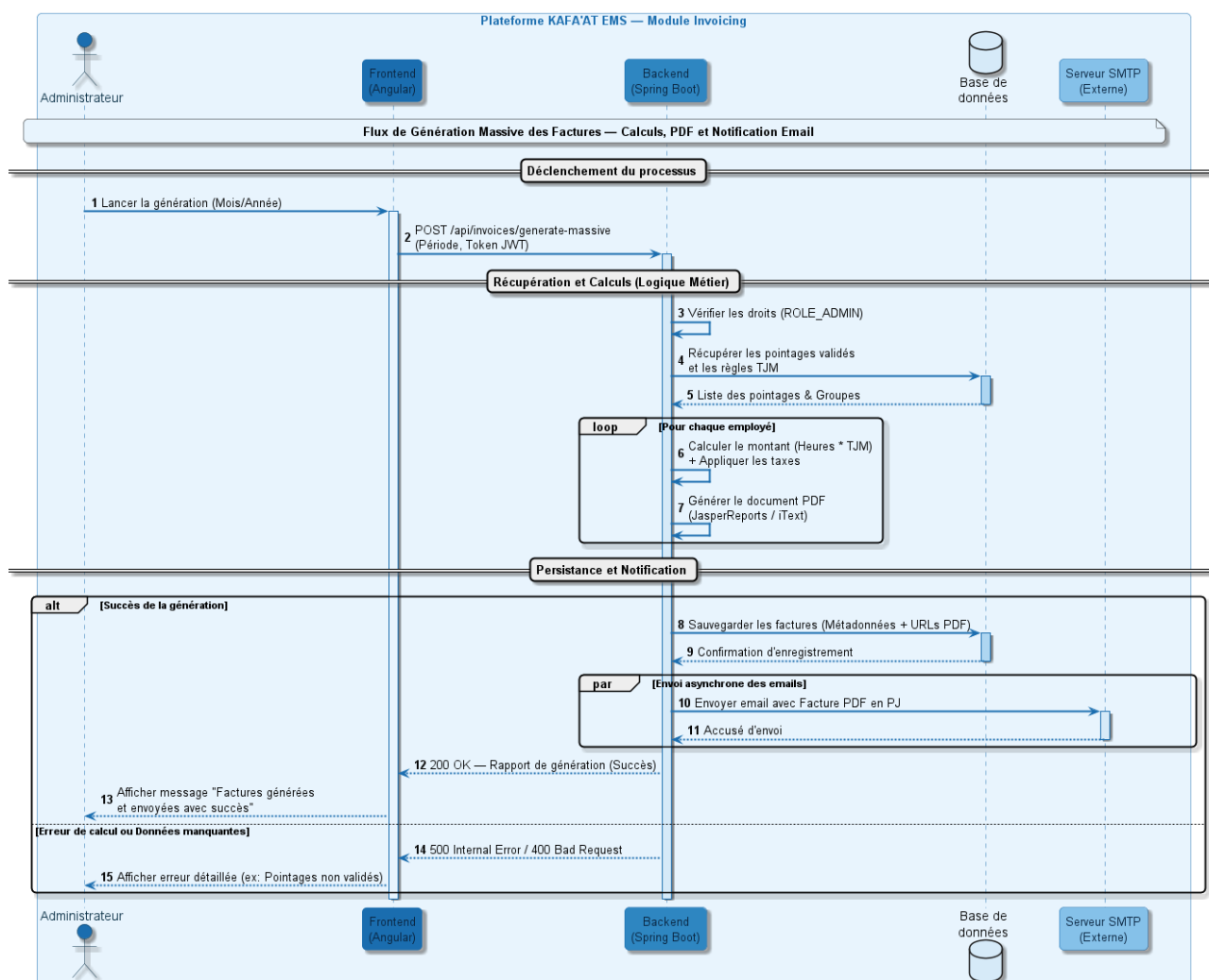


FIGURE 2.10 – Diagramme de séquence de la génération massive des factures

2.2.3.4 Diagramme de séquence « Désactiver un compte utilisateur »

Dans un système de gestion financière et de suivi du temps de travail, la suppression définitive d'un utilisateur (Hard Delete) est proscrite afin de préserver l'intégrité de l'historique des factures et des pointages. Par conséquent, nous avons opté pour un mécanisme de **Soft Delete** (désactivation logique).

Ce diagramme met en évidence le contrôle d'accès basé sur les rôles (RBAC) mis en place au niveau de Spring Security. Lorsqu'une requête de désactivation est émise, le **Backend** vérifie impérativement que le jeton JWT transmis contient le rôle **ROLE_ADMIN** avant de procéder à la mise à jour du statut du compte (**isActive = false**) dans la base de données.

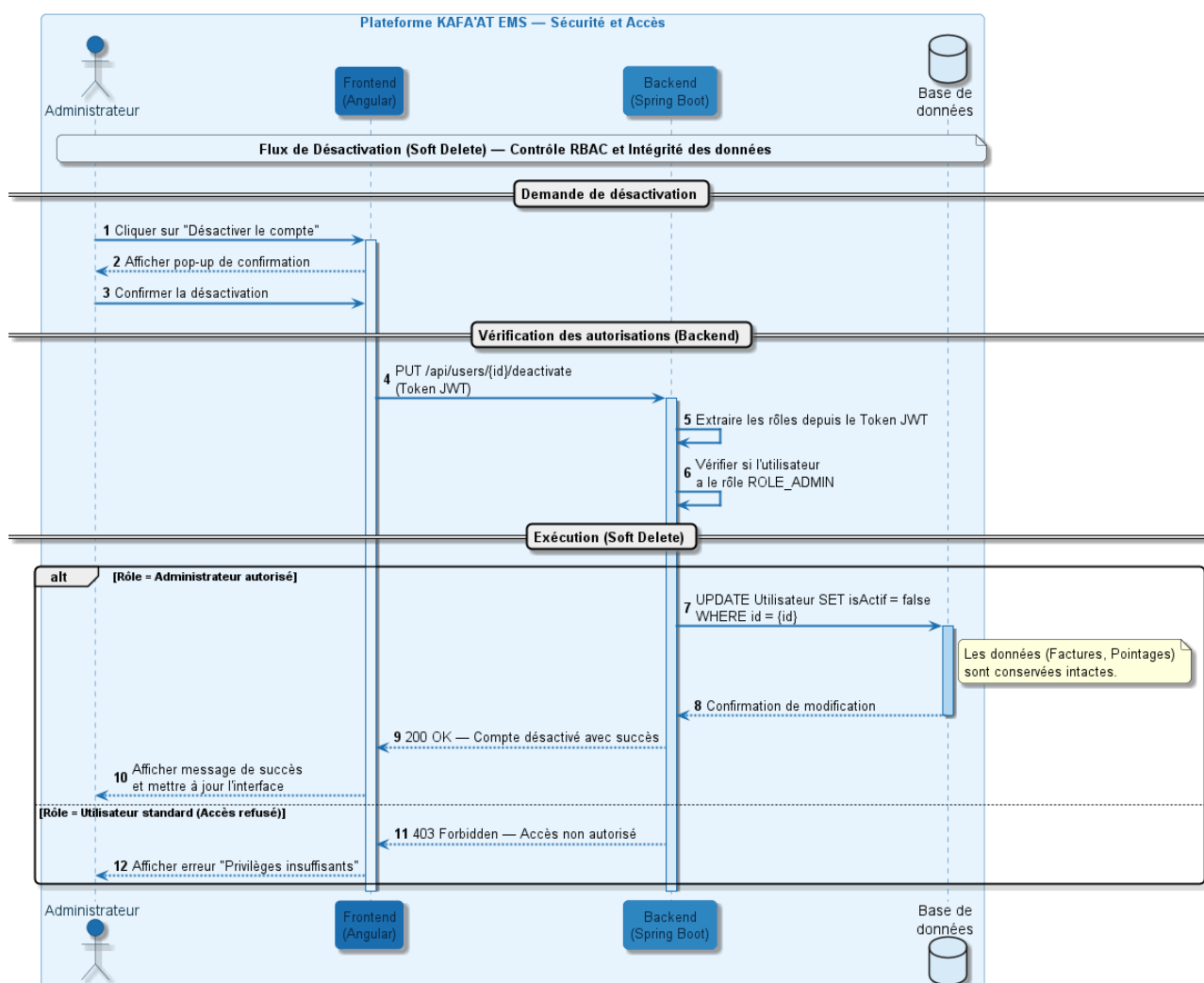


FIGURE 2.11 – Diagramme de séquence de la désactivation d'un compte (Soft Delete)

2.2.3.5 Diagramme de séquence « Créer un groupe de facturation »

La configuration des groupes de facturation est un prérequis indispensable pour le fonctionnement du moteur d'invoicing. C'est à travers cette interface que l'administrateur définit les règles de tarification (Taux Journalier Moyen, Devise, Plafond d'heures) qui seront appliquées aux collaborateurs.

Ce diagramme de séquence illustre le processus d'ajout d'un nouveau groupe. Au-delà de la simple insertion de données, le **Backend** implémente une logique de validation métier pour garantir l'intégrité de la base de données. Il vérifie notamment les privilèges de l'utilisateur (ROLE_ADMIN) et s'assure qu'aucun groupe portant le même nom n'existe déjà, évitant ainsi les doublons et les conflits financiers (409 Conflict).

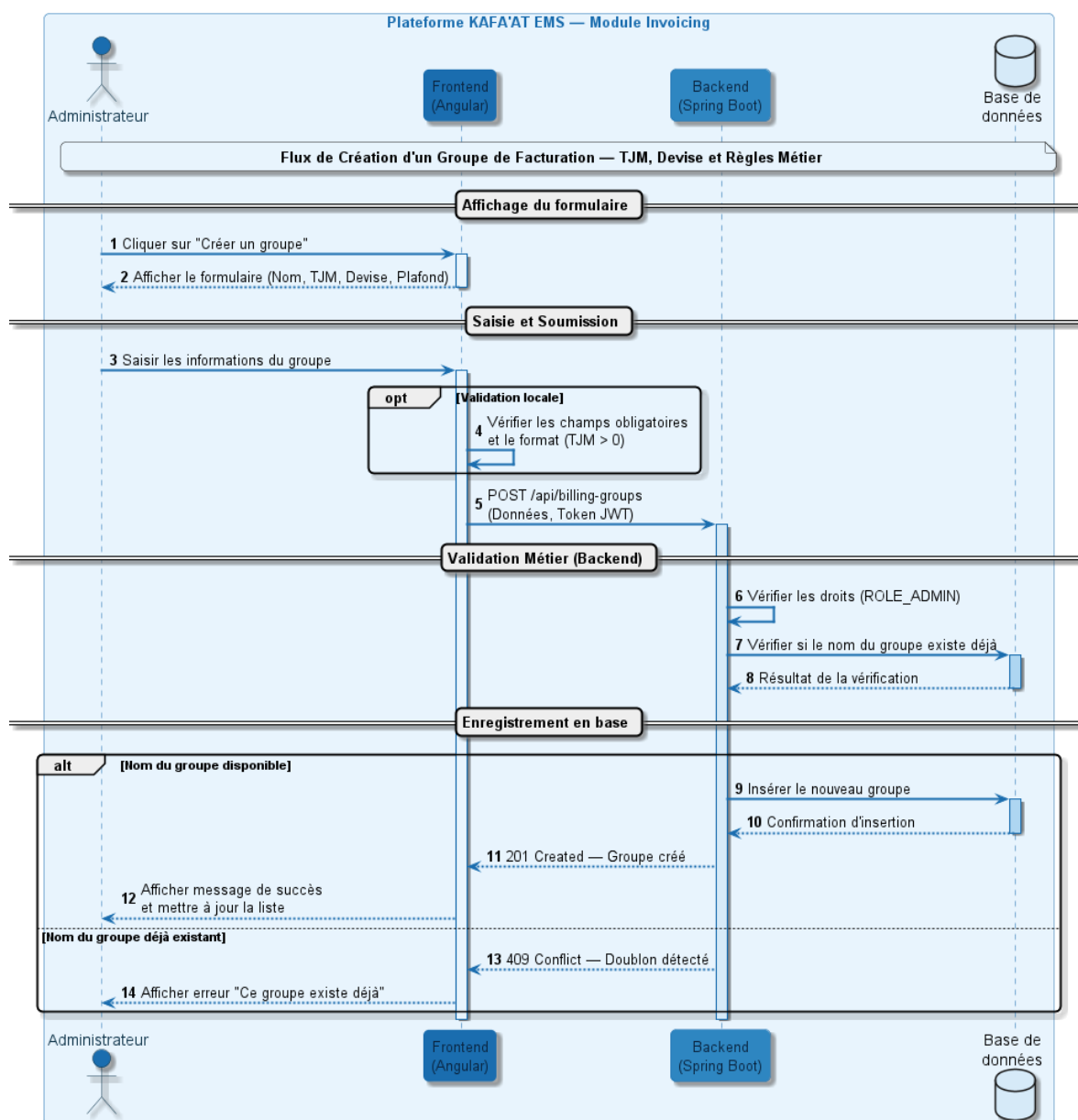


FIGURE 2.12 – Diagramme de séquence de la création d'un groupe de facturation

2.2.3.6 Diagramme de séquence « Appliquer un ajustement »

Afin de garantir une traçabilité totale et une transparence financière au sein de la plateforme, la modification des heures de travail déclarées par un employé n'est pas une simple mise à jour en base de données.

Lorsqu'un administrateur constate une anomalie et décide d'appliquer un ajustement (Override) sur un pointage, ce diagramme de séquence montre la rigueur du processus métier implémenté. Le système exige obligatoirement la saisie d'un **motif justificatif**. Une fois l'ajustement historisé en base de données de manière immuable, le **Backend** déclenche l'envoi d'une alerte automatique via le **serveur SMTP** pour notifier l'employé concerné de la modification de sa feuille de temps.

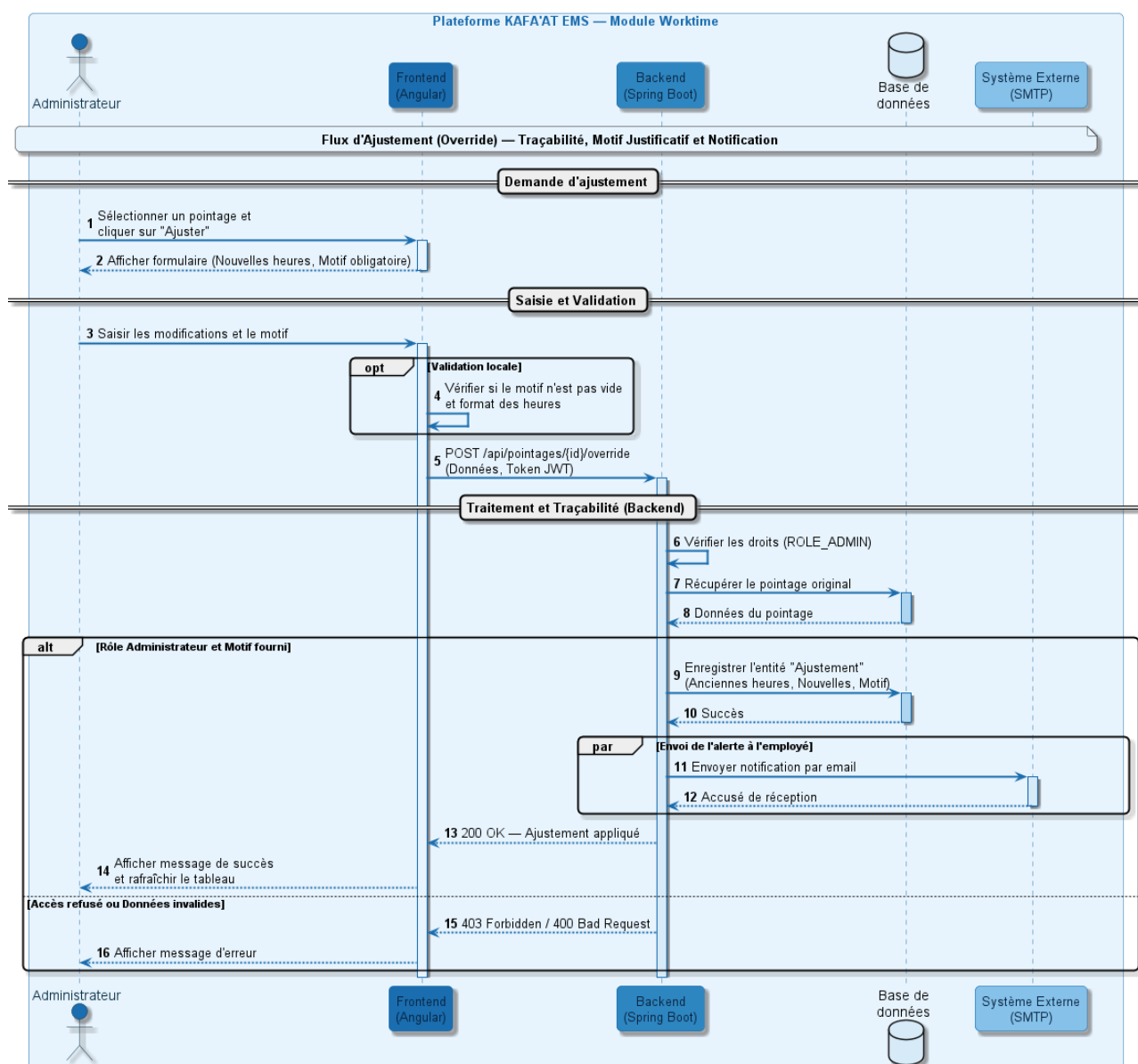


FIGURE 2.13 – Diagramme de séquence d'un ajustement de pointage (Override)

2.2.3.7 Diagramme de séquence « Gestion détaillée d'une facture »

La gestion d'une facture générée n'est pas une simple opération de mise à jour, mais englobe tout le cycle de vie du document financier. Ce diagramme illustre un scénario complexe où l'administrateur interagit avec plusieurs fonctionnalités liées à une même facture.

Le processus s'amorce par une modification des informations fondamentales de la facture (telle que l'application d'une remise commerciale exceptionnelle, l'ajustement ponctuel du TJM, ou la révision de la date d'échéance). Une fois ces données de base validées et persistées, l'architecture du système, conçue pour être hautement modulaire, prévoit des blocs d'exécution optionnels (opt). Ces blocs offrent à l'administrateur une gestion granulaire du cycle de vie du document financier, lui permettant de :

- **Gérer les transitions d'état financier :** Mettre à jour le statut de la facture (par exemple, la faire passer de *EN_ATTENTE* à *PAYÉE*) lors de la confirmation du règlement par le client. Cette action est vitale pour la cohérence des tableaux de bord de suivi et permet de verrouiller la facture contre toute modification ultérieure.
- **Déclencher la régénération dynamique du document :** Initier la création d'un nouveau document PDF de manière asynchrone. Si les montants, les taxes ou les remises ont été altérés lors de l'étape précédente, le système doit impérativement recalculer les totaux et écraser l'ancienne version. Cela garantit une stricte conformité entre les données relationnelles (Base de données) et le document physique archivé.
- **Assurer une récupération sécurisée du fichier :** Interroger l'API du serveur pour télécharger le fichier PDF final. Le Backend, après avoir vérifié les habilitations via le Token JWT, renvoie le fichier sous forme de flux binaire (*Blob*). Cette approche optimise la consommation de la bande passante et sécurise l'accès aux documents sensibles en évitant l'exposition d'URL publiques statiques.

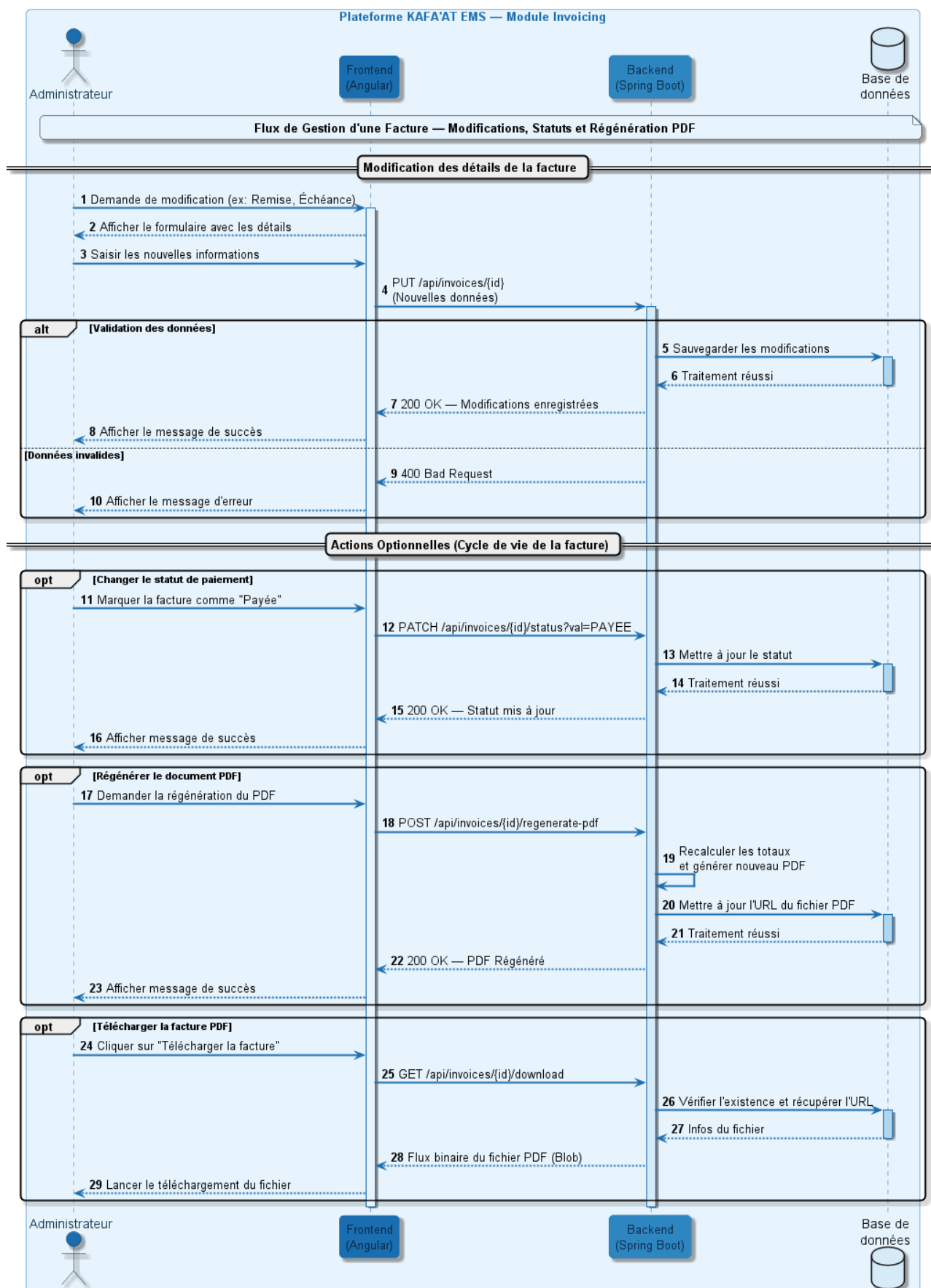


FIGURE 2.14 – Diagramme de séquence de la gestion détaillée d'une facture

2.2.4 Diagrammes d'activité

Les diagrammes d'activité sont des diagrammes utilisés dans le langage de modélisation unifié (UML) pour représenter le flux d'activités ou de processus dans un système. Ils fournissent une vue visuelle de l'ordre des actions et des décisions prises au sein d'un système, en mettant l'accent sur les activités réalisées et les transitions entre ces activités. Les diagrammes d'activité sont constitués de nœuds d'activité, de transitions et de décisions. Les nœuds d'activité représentent les actions ou les étapes réalisées dans le système, tandis que les transitions indiquent le passage d'une activité à une autre. Les décisions permettent de représenter les points de branchement conditionnels dans le flux d'activité.

Ces diagrammes sont particulièrement utiles pour modéliser des processus métier, des algorithmes ou des flux de contrôle dans un système. Ils permettent de visualiser le déroulement des activités et de comprendre les différentes étapes et décisions qui sont prises tout au long du processus. [11]

Dans le contexte de notre plateforme **KAFA'AT EMS**, ces diagrammes nous permettront de détailler avec précision la logique interne de nos processus clés, tels que la soumission et la validation des pointages, ainsi que le cycle complet de facturation.

2.2.4.1 Diagramme d'activité « Validation des pointages »

Ce premier diagramme d'activité modélise la logique métier et le flux de contrôle relatifs au suivi du temps de travail (Worktime). Afin d'illustrer clairement les responsabilités de chaque acteur, nous avons structuré ce diagramme en **couloirs d'activités (Swimlanes)** séparant l'Employé, le Système (Backend) et l'Administrateur.

L'algorithme se divise en deux phases critiques :

1. **Contrôle à la soumission** : Le système agit comme un premier filtre. Il vérifie de manière autonome si le cumul des heures déclarées par l'employé ne dépasse pas le plafond journalier autorisé. Si la condition est respectée, le pointage est enregistré sous le statut *SOUMIS*.
2. **Arbre de décision administratif** : Lors de la révision, l'administrateur dispose de trois branches conditionnelles : valider, refuser (avec motif) ou ajuster (Override). Chaque décision déclenche un comportement distinct dans le système, incluant la mise à jour des statuts et l'envoi de notifications asynchrones.

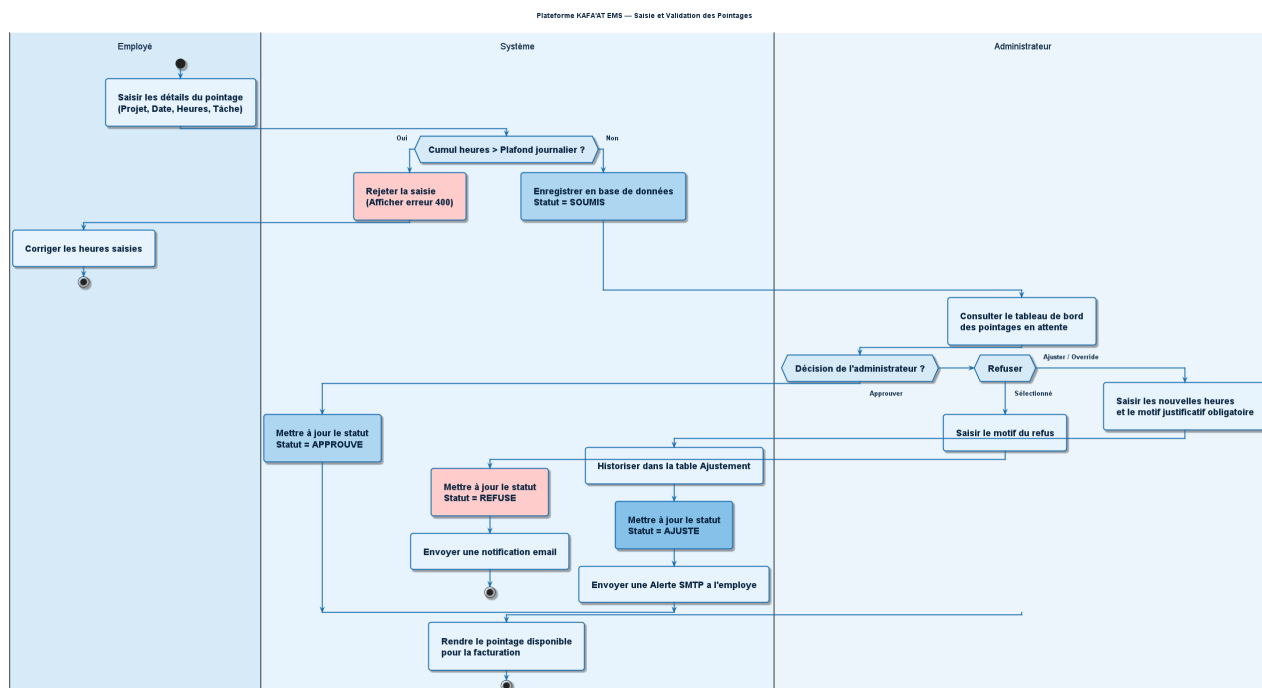


FIGURE 2.15 – Diagramme d’activité avec Swimlanes : Validation des pointages

2.2.4.2 Diagramme d’activité « Génération Massive des Factures »

Ce deuxième diagramme d’activité illustre le cœur algorithmique de notre module d’Invoicing. La génération massive des factures est une opération critique qui exige un traitement par lots (Batch processing) optimisé et une gestion rigoureuse des exceptions.

Le processus débute par une vérification stricte des habilitations de l’administrateur via son Token JWT. Le système extrait ensuite les pointages marqués comme *APPROUVÉS*. L’algorithme principal repose sur une **structure itérative** qui traite chaque employé individuellement. Afin de prévenir les blocages de la chaîne d’exécution, le système intègre une gestion d’erreur interne : si un employé ne possède pas de paramètres financiers configurés (Groupe de Facturation/TJM manquant), une anomalie est journalisée et le système passe à l’itération suivante sans interruption.

Pour les profils valides, le moteur calcule les montants (HT et TTC), génère le document PDF via **JasperReports**, et prépare l’entité. Pour des raisons de performance, l’enregistrement en base de données s’effectue via une insertion en masse (*Batch Insert*).

Enfin, le flux d’exécution se scinde (*Fork*) : l’envoi des courriels contenant les pièces jointes est délégué à un processus asynchrone, tandis que l’interface administrateur affiche instantanément un tableau de bord de synthèse détaillant le nombre de factures générées, le chiffre d’affaires total, et les éventuelles anomalies rencontrées.

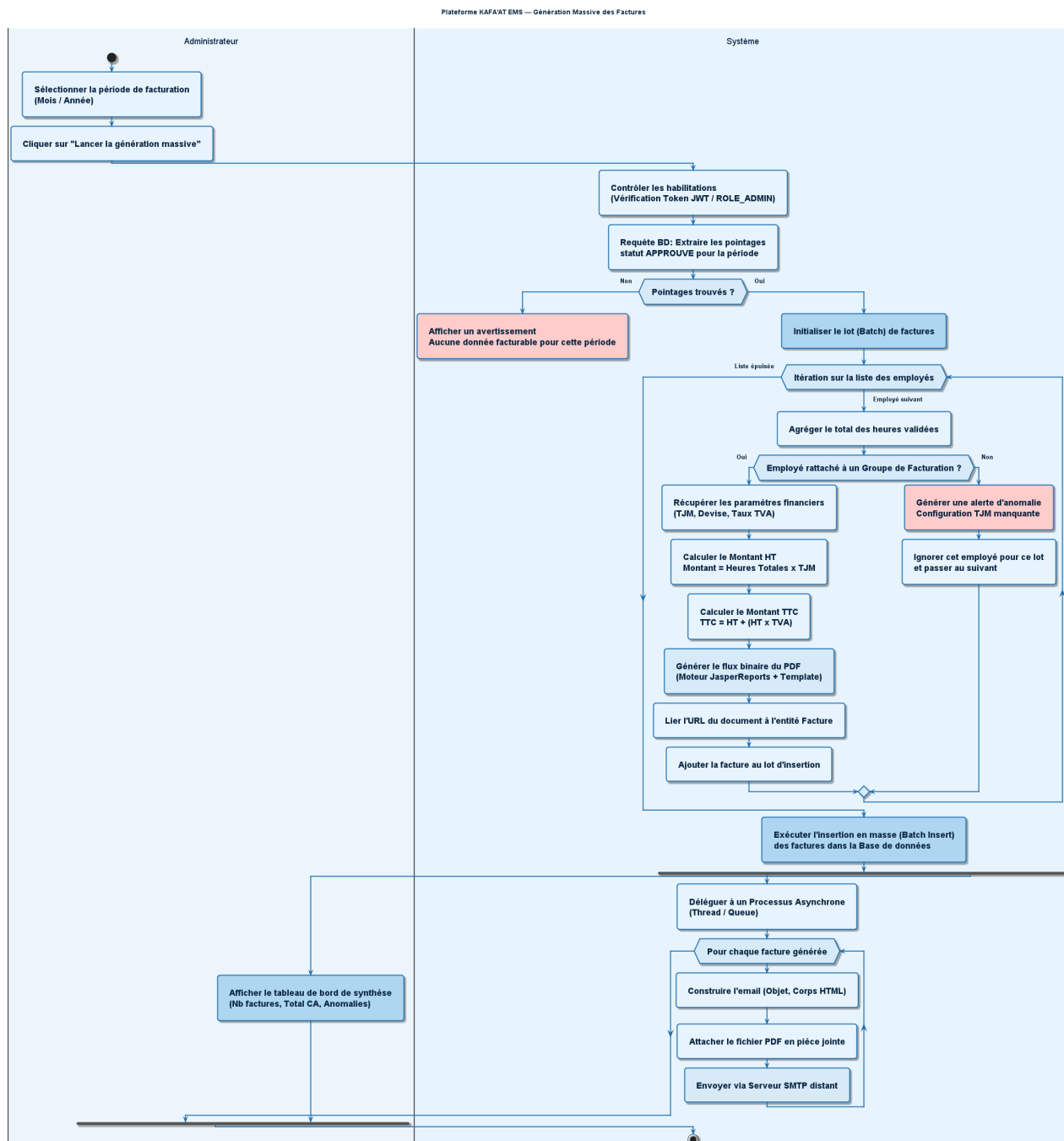


FIGURE 2.16 – Diagramme d'activité détaillé : Moteur de génération massive des factures

2.2.4.3 Diagramme d'activité « Suivi des factures »

Ce dernier diagramme d'activité modélise le cycle de vie post-génération d'une facture, englobant les opérations de suivi, d'encaissement et d'exportation. Il illustre les différentes branches décisionnelles qui s'offrent à l'administrateur lors de la consultation du module d'Invoicing.

Le processus débute par l'accès à la liste des factures. Si aucune facture spécifique n'est sélectionnée, l'administrateur a la possibilité de déclencher une extraction globale des données financières sous forme de fichier **Excel/CSV** pour un traitement externe.

Lorsqu'une facture est sélectionnée, le système oriente le flux selon l'action choisie :

- **Recouvrement** : Enregistrement des références de paiement, entraînant une mise à jour du statut vers *PAYÉE* et la génération éventuelle d'un reçu.
- **Vérification des échéances** : Le système effectue un contrôle logique. Si la facture est toujours en attente et que la date actuelle dépasse la date d'échéance convenue, le statut bascule automatiquement en *EN_RETARD* et une relance par courrier électronique est déclenchée.
- **Consultation** : Téléchargement direct du document PDF archivé.

Indépendamment de l'action spécifique effectuée (modification ou consultation), le système clôture le processus en consignait l'opération dans un journal d'audit afin d'assurer une traçabilité complète des manipulations financières.

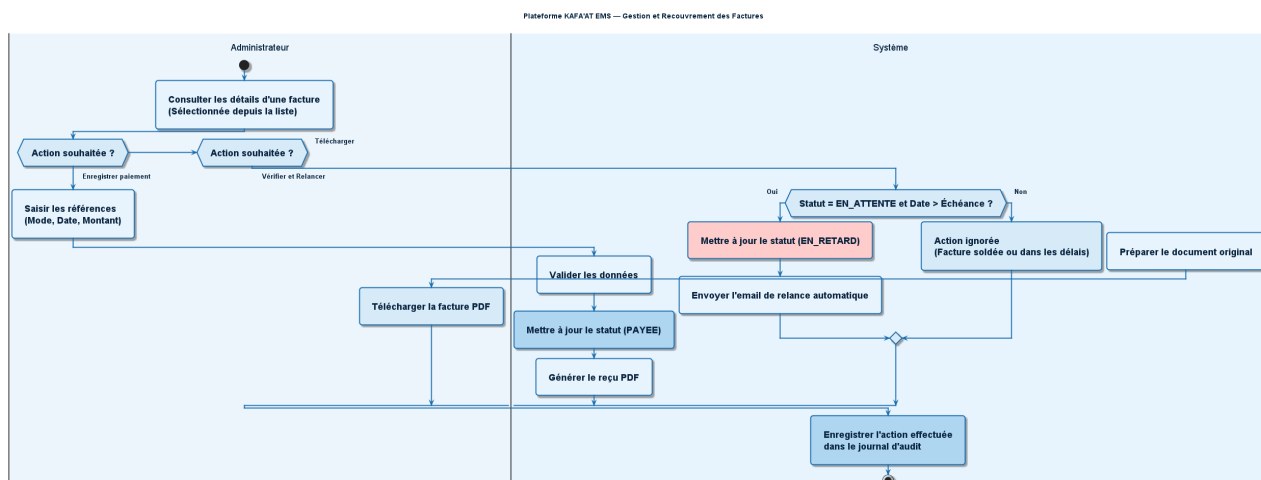


FIGURE 2.17 – Diagramme d'activité : Gestion, relances et recouvrement des factures

Conclusion

Dans ce chapitre, nous avons détaillé la phase de conception logicielle de notre projet en nous appuyant sur les standards du langage de modélisation UML. À travers l'élaboration progressive des diagrammes de cas d'utilisation, de classes, de séquence et d'activité, nous avons pu traduire les exigences métiers complexes des modules *Invoicing* et *Worktime* en spécifications techniques claires et rigoureuses.

Cette démarche de modélisation approfondie nous a permis de structurer une architecture robuste pour la plateforme KAFA'AT EMS. Nous avons ainsi pu anticiper la logique des processus critiques, tels que la validation hiérarchique des pointages, le moteur de génération massive des factures, et la gestion des habilitations, tout en garantissant la traçabilité et l'intégrité des données financières.

La conception conceptuelle et dynamique étant à présent achevée, ces modèles constitueront notre feuille de route pour la phase de développement. Le chapitre suivant sera ainsi consacré à la réalisation de l'application, où nous mettrons en œuvre cette architecture à travers l'implémentation des technologies choisies, notamment **Spring Boot** pour le Backend et **Angular** pour le Frontend.

Chapitre 3

Réalisation et Mise en œuvre

Introduction

Dans ce chapitre, nous abordons la concrétisation technique et la mise en œuvre de notre solution logicielle au sein de la plateforme KAFA'AT EMS. Nous commencerons par définir l'architecture globale du système, en justifiant les choix technologiques adoptés pour répondre aux exigences de performance, de sécurité et de scalabilité. Ensuite, nous détaillerons l'environnement de développement et les outils mis à profit lors de la réalisation. Enfin, nous présenterons les interfaces utilisateurs développées, en mettant l'accent sur l'ergonomie et l'expérience utilisateur (UX). Ce chapitre constitue ainsi la passerelle concrète entre la modélisation théorique et le produit final déployable.

3.1 Architecture de l'application

L'architecture logicielle structure le système en définissant ses composants, leurs responsabilités et leurs interactions. Pour les modules *Invoicing* et *Worktime* de la plateforme KAFA'AT EMS, nous avons adopté une approche multicouche, modulaire et hautement découplée. Cette configuration assure une stricte séparation des préoccupations (*Separation of Concerns*) tout en optimisant la maintenance évolutive et la sécurité des flux.

La figure 3.1 illustre cette architecture globale, mettant en évidence la communication fluide entre la couche de présentation (Angular), la logique métier centralisée (Spring Boot) et la persistance des données (PostgreSQL).

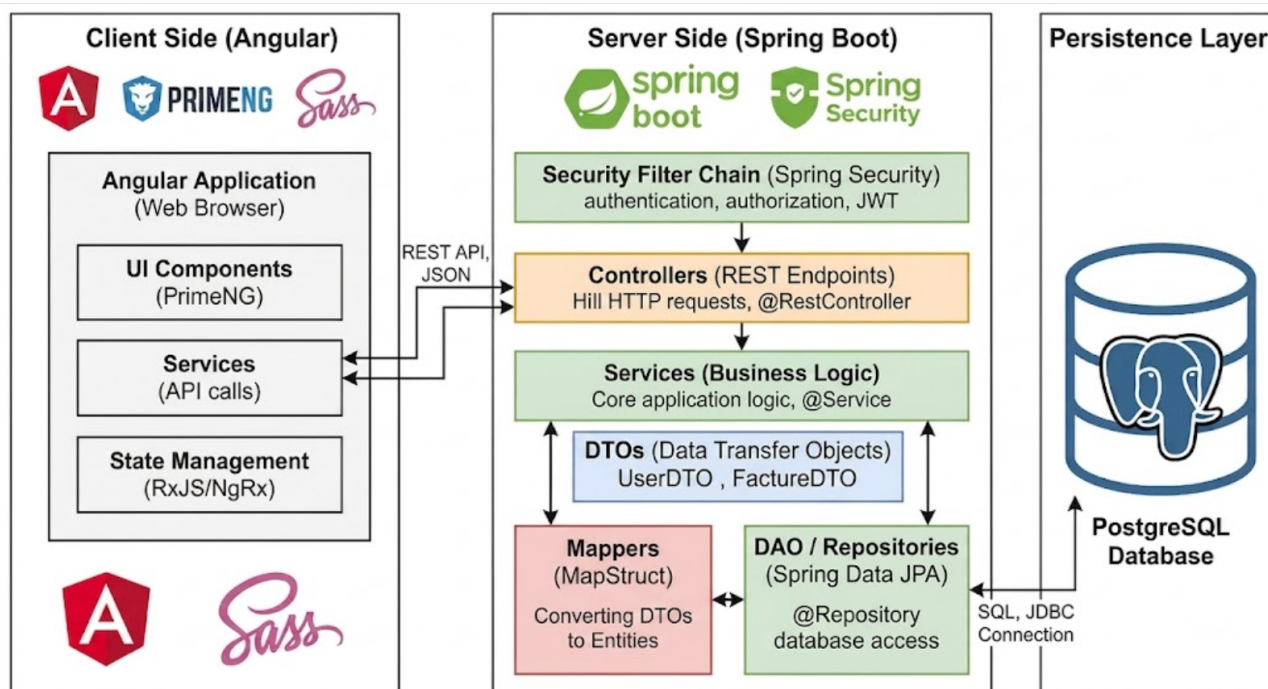


FIGURE 3.1 – Architecture globale de l'application

3.1.1 Architecture Backend

Pour la partie serveur, nous avons opté pour le framework **Spring Boot** selon une architecture en couches (N-Tier). Cette approche isole la logique métier, sécurise les échanges et facilite la maintenance. Les couches s'articulent ainsi :

- **Sécurité (Spring Security & JWT)** : Intercepte les requêtes pour assurer l'authentification et l'autorisation via les tokens JWT.
- **API REST (Controllers)** : Expose les points d'entrée, valide les requêtes HTTP du client Angular et les achemine vers les services.
- **Couche Métier (Services)** : Cœur de l'application, elle encapsule les algorithmes complexes (*Invoicing, Worktime*).
- **Objets de Transfert (DTOs & MapStruct)** : Limite les données exposées en convertissant automatiquement les Entités en DTOs.
- **Accès aux Données (Spring Data JPA)** : Gère la persistance et les transactions avec la base de données **PostgreSQL**.
- **Génération (JasperReports)** : Dédiee à la création dynamique des factures au format PDF.

Cette structuration rigoureuse garantit un code propre, testable et prêt à évoluer au sein de la plateforme KAFA'AT EMS.

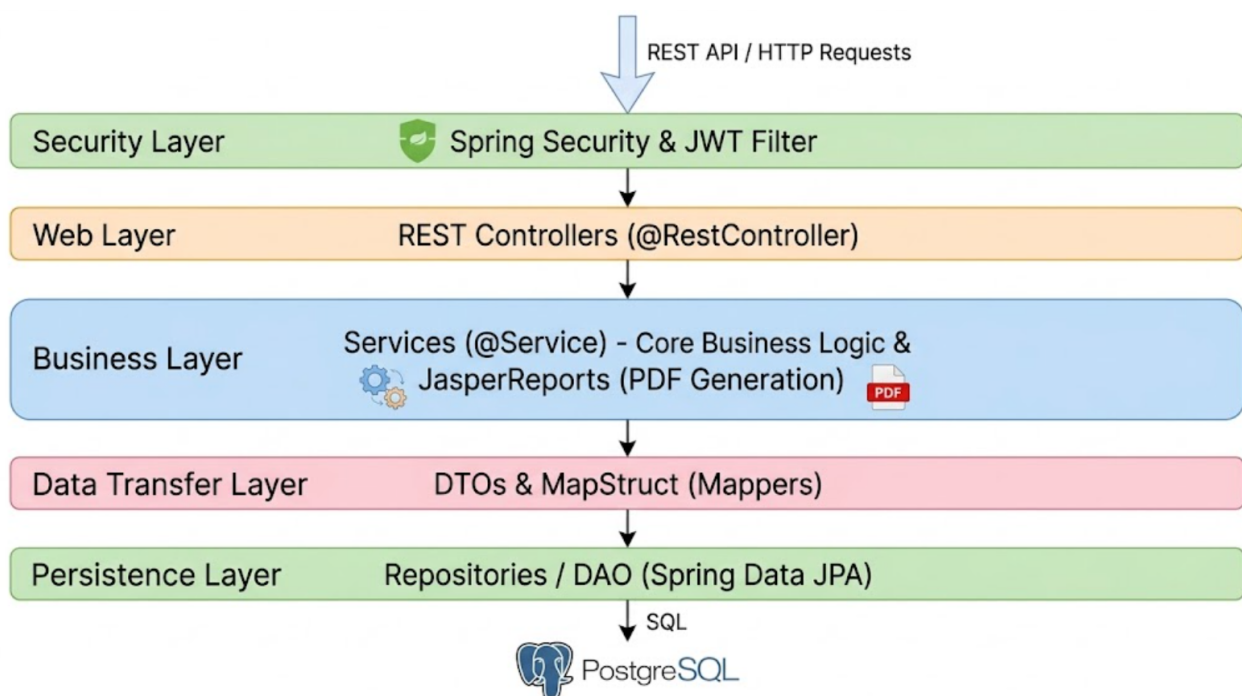


FIGURE 3.2 – Architecture en couches du Backend (Spring Boot)

3.1.2 Architecture Frontend

Pour la partie cliente, notre application repose sur **Angular**, structuré selon une architecture orientée composants (Component-Based Architecture). Ce choix, couplé à **TypeScript** et à la bibliothèque graphique **PrimeNG**, garantit une interface (UI) réactive et modulaire pour les modules *Invoicing* et *Worktime*. L'architecture s'organise ainsi :

- **Composants & Templates (PrimeNG / SCSS)** : Blocs de construction de l'interface. Chaque composant encapsule sa logique métier, sa structure HTML et son style, en intégrant les éléments riches de PrimeNG.
- **Services & API Client** : Classes encapsulant les appels HTTP vers le backend (Spring Boot), assurant la réutilisabilité et la stricte séparation des préoccupations.
- **Routage (SPA Routing)** : Gère la navigation *Single Page Application*, permettant des transitions fluides et sécurisées entre les différentes vues de la plateforme.
- **Programmation Réactive (RxJS)** : Gestion asynchrone des flux de données (Observables) pour une mise à jour dynamique de l'interface en temps réel.
- **Modules & Lazy Loading** : Découpage fonctionnel du code permettant le chargement différé des ressources, optimisant ainsi les performances et le temps de réponse initial.

Cette architecture modulaire garantit une interface réactive et facilite la maintenance de la plateforme KAFA'AT EMS, comme illustré ci-dessous.

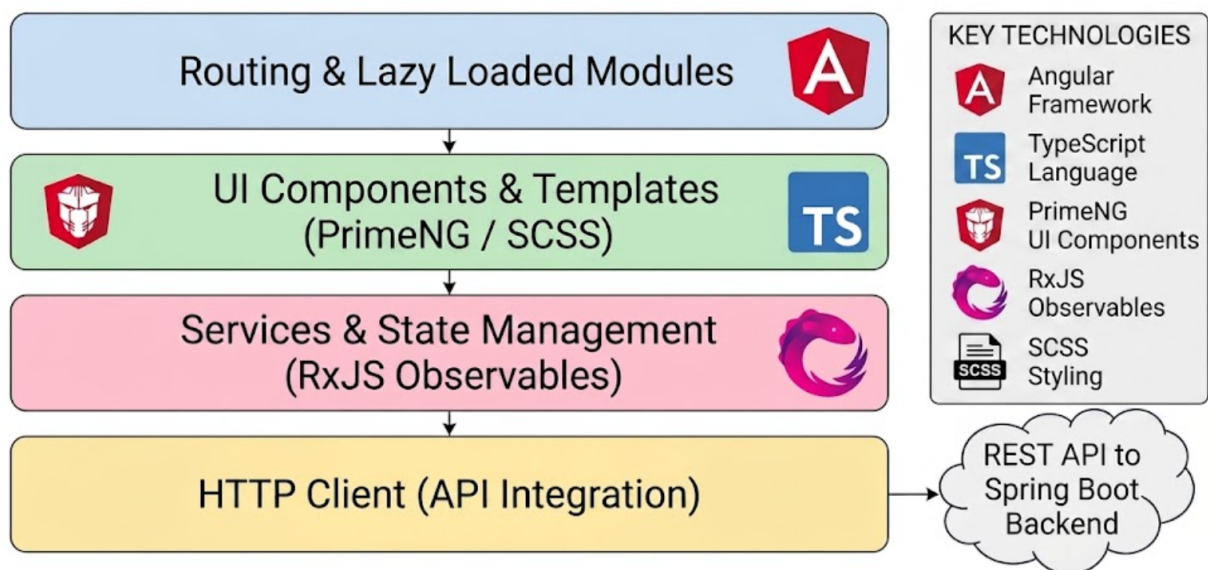


FIGURE 3.3 – Architecture interne de l'application cliente (Angular)

3.2 Technologies et outils de développement

Cette section détaille l'écosystème technologique mis en place pour la réalisation de notre projet. Le choix de ces outils repose sur des critères stricts de performance, de sécurité et d'adéquation avec les standards de l'industrie logicielle.

3.2.1 Environnements logiciels

3.2.1.1 IntelliJ IDEA



FIGURE 3.4 – Logo d'IntelliJ IDEA

IntelliJ IDEA a été sélectionné comme Environnement de Développement Intégré (IDE) principal pour la conception et l'implémentation de la couche backend (Spring Boot).

Reconnu pour sa robustesse, cet outil a considérablement optimisé notre cycle de développement grâce à son analyse statique de code avancée, son autocomplétion intelligente et son intégration native avec l'écosystème Java. Il a particulièrement facilité le refactoring du code, la gestion des dépendances et le débogage complexe des algorithmes liés aux processus de facturation.

3.2.1.2 Visual Studio Code



FIGURE 3.5 – Logo de Visual Studio Code

Visual Studio Code (VS Code) a été adopté comme éditeur de choix pour l'implémentation de la partie frontend (Angular) de notre application.

Sa légèreté, couplée à un écosystème d'extensions extrêmement riche, a permis d'adapter l'environnement aux exigences spécifiques de notre *Stack* technologique. L'intégration native de terminaux multiples a grandement facilité l'exécution continue des commandes *Angular CLI*. De plus, les extensions dédiées à TypeScript, le formatage automatique (Prettier) et l'intégration fluide de Git ont assuré un développement rapide, tout en maintenant une qualité de code optimale et une gestion efficace des *Pull Requests*.

3.2.1.3 Swagger (OpenAPI)



FIGURE 3.6 – Logo de Swagger

Swagger a été intégré à notre backend Spring Boot afin de générer automatiquement une documentation interactive et standardisée de notre API REST, basée sur la spécification OpenAPI.

Cet outil s'est avéré indispensable lors de la phase de développement pour l'exploration et le test direct des *Endpoints* (notamment ceux liés aux processus d'Invoicing et de Worktime) sans nécessiter de client externe. En fournissant un contrat d'interface clair, Swagger a considérablement fluidifié l'intégration logicielle et la communication technique entre notre logique serveur et les services clients de l'application Angular.

3.2.1.4 Apache Tomcat



FIGURE 3.7 – Logo d'Apache Tomcat

Apache Tomcat est un conteneur de servlets et un serveur web open-source de référence pour l'environnement Java. Dans le cadre de notre projet, Tomcat a été utilisé dans sa version embarquée (*embedded*) nativement fournie par le framework Spring Boot.

Cette intégration transparente nous a affranchis des configurations fastidieuses liées au déploiement d'un serveur externe. Elle a permis de lancer, de tester et de déployer nos API REST (gérant la logique complexe des modules *Invoicing* et *Worktime*) de manière totalement autonome, légère et performante tout au long de notre cycle de développement.

3.2.1.5 Docker



FIGURE 3.8 – Logo de Docker

Afin de garantir un environnement de développement et de déploiement standardisé, nous avons intégré Docker, une plateforme de conteneurisation incontournable. Son utilisation s'est articulée autour de trois axes majeurs au sein de notre projet :

- **Base de données locale (*Docker Compose*)** : Instanciation rapide de notre base de données PostgreSQL via un fichier `docker-compose.yml`, éliminant ainsi le besoin d'installations lourdes sur la machine locale.
- **Conteneurisation de l'application (*Dockerfile*)** : Création d'images sur mesure pour le backend (Spring Boot) et le frontend (Angular), garantissant une isolation parfaite et une portabilité totale des livrables.

- **Intégration Continue (CI/CD) :** Alignement avec les pratiques *DevOps* de l'équipe NTT DATA, en facilitant la création et le déploiement automatisé de la plateforme KAFA'AT EMS sur les environnements de test et de production.

Cette approche moderne a considérablement optimisé notre flux de travail, résolvant les problèmes classiques de compatibilité inter-environnements et apportant une robustesse de niveau entreprise à l'ensemble du système.

3.2.1.6 Apache Maven



FIGURE 3.9 – Logo d'Apache Maven

Apache Maven a été utilisé comme outil principal d'automatisation de compilation et de gestion des dépendances pour notre backend Spring Boot.

Via son fichier de configuration central `pom.xml`, Maven a structuré le projet et géré de manière transparente l'intégration des bibliothèques tierces indispensables à notre logique métier (telles que *Spring Security* pour l'authentification JWT, *MapStruct* pour le mappage des DTOs, et *JasperReports* pour la facturation). Il a automatisé notre processus de *build*, garantissant ainsi une construction fiable, reproductible et prête pour l'intégration continue.

3.2.1.7 Node.js



FIGURE 3.10 – Logo de Node.js

Bien que notre logique serveur soit gérée par Spring Boot, l'environnement Node.js s'est avéré indispensable pour la partie cliente de notre application. Dans notre architecture, Node.js n'agit pas comme un serveur web de production, mais exclusivement comme moteur d'exécution en arrière-plan pour l'écosystème de développement frontend. Il fournit l'environnement nécessaire au fonctionnement d'Angular CLI, permettant la compilation à la volée du code TypeScript, la minification des ressources et le lancement du serveur de développement local.

3.2.1.8 NPM (Node Package Manager)



FIGURE 3.11 – Logo de NPM

NPM intervient comme le gestionnaire de paquets standard pour notre couche frontend, jouant un rôle symétrique à celui de Maven pour le backend. Il a facilité la résolution, l'installation et la gestion des versions de l'ensemble des dépendances Angular requises par notre interface. Via le fichier de configuration `package.json`, NPM nous a permis d'intégrer efficacement des bibliothèques clés pour l'ergonomie de l'application, telles que le framework UI *PrimeNG* pour la création des composants visuels, et *RxJS* pour la gestion réactive des événements.

3.2.1.9 JWT (JSON Web Token)



FIGURE 3.12 – Logo de JWT

Dans le cadre de la sécurisation de notre architecture distribuée, JWT a été adopté comme standard pour gérer l'authentification et l'autorisation entre notre client Angular et l'API REST Spring Boot. Contrairement aux sessions classiques, JWT nous a permis de mettre en place une approche *stateless* (sans état). Une fois l'utilisateur authentifié, le jeton généré encapsule ses informations et ses rôles sous forme chiffrée. Il est ensuite transmis via les en-têtes HTTP (*Headers*), garantissant ainsi un accès hautement sécurisé aux ressources sensibles des modules *Invoicing* et *Worktime* sans surcharger la mémoire du serveur.

3.2.1.10 Postman



FIGURE 3.13 – Logo de Postman

Postman s'est imposé comme l'outil de prédilection pour le test, le débogage et la validation de nos API REST. Indépendamment du développement de l'interface graphique (Angular), Postman nous a permis d'interroger directement les *endpoints* exposés par notre backend Spring Boot.

Grâce à cet outil, nous avons pu simuler des requêtes HTTP complexes, valider la conformité des structures de données JSON (DTOs) entrantes et sortantes liées aux modules *Invoicing* et *Worktime*. De plus, Postman a été crucial pour tester notre couche de sécurité, en nous permettant d'injecter manuellement les jetons JWT dans les en-têtes d'autorisation (*Authorization Headers*) afin de vérifier les restrictions d'accès selon les rôles utilisateurs.

3.2.1.11 PlantUML



FIGURE 3.14 – Logo de PlantUML

Pour la phase d’analyse et de conception (détaillée dans le chapitre précédent), nous avons délibérément écarté les logiciels de modélisation graphique classiques au profit de PlantUML. Cet outil open-source permet de générer des diagrammes UML (cas d’utilisation, séquences, activités) à partir d’un langage de description textuel clair. Cette approche novatrice de « *Diagram as Code* » a offert des avantages considérables : elle a permis un versionnage fluide de nos modèles via Git, facilité les revues lors des *Pull Requests*, et garanti une esthétique uniforme et professionnelle pour l’ensemble des schémas architecturaux du projet.

3.2.2 Frameworks et technologies clés

3.2.2.1 Spring Boot



FIGURE 3.15 – Logo de Spring Boot

Spring Boot constitue l’épine dorsale de notre architecture backend. Ce framework a été privilégié pour sa capacité à amorcer rapidement des applications Java robustes grâce à son mécanisme d’auto-configuration et son conteneur d’inversion de contrôle (IoC). Au sein de la plateforme KAFA’AT EMS, il nous a permis de nous affranchir des configurations fastidieuses pour nous concentrer exclusivement sur la logique métier complexe des modules *Invoicing* et *Worktime*. Il a grandement facilité l’implémentation et l’exposition de nos services via des API RESTful performantes, autonomes et prêtes pour la production.

3.2.2.2 Spring Security



FIGURE 3.16 – Logo de Spring Security

Spring Security est le module incontournable que nous avons intégré pour assurer la protection globale de notre backend. Contrairement aux approches monolithiques traditionnelles basées sur les sessions, nous l’avons configuré pour opérer de manière *stateless* (sans état) en parfaite synergie avec notre système de jetons JWT. Ce framework nous a permis d’implémenter un

contrôle d'accès granulaire basé sur les rôles (RBAC). Ainsi, il garantit que chaque requête HTTP ciblant des ressources sensibles (comme les données de facturation ou les pointages) soit rigoureusement interceptée, authentifiée et autorisée avant d'atteindre nos contrôleurs REST.

3.2.2.3 Spring Data JPA



FIGURE 3.17 – Logo de Spring Data JPA

Pour la couche de persistance, nous avons exploité Spring Data JPA, soutenu en arrière-plan par l'ORM Hibernate. Ce composant stratégique nous a permis d'abstraire la complexité des requêtes SQL et d'interagir avec notre base de données PostgreSQL via une approche orientée objet. En étendant simplement les interfaces `JpaRepository`, nous avons drastiquement réduit le code *boilerplate* pour les opérations CRUD. De plus, Spring Data JPA a assuré une gestion transactionnelle rigoureuse, un aspect critique pour maintenir l'intégrité des données financières dans le module *Invoicing* et l'exactitude des pointages dans le module *Worktime*.

3.2.2.4 Angular



FIGURE 3.18 – Logo d'Angular

Côté client, Angular s'est imposé comme le framework de choix pour concevoir l'interface utilisateur de notre plateforme. En s'appuyant sur la robustesse du typage strict de TypeScript, il nous a permis de structurer une *Single Page Application* (SPA) modulaire. Grâce à sa gestion efficace de l'état asynchrone et à l'intégration de bibliothèques tierces comme *PrimeNG*, Angular a offert une expérience utilisateur fluide et hautement réactive. Il a servi de pont direct avec notre backend, consommant efficacement les API REST sécurisées par JWT pour afficher et manipuler dynamiquement les tableaux de bord et les interfaces de gestion.

3.2.2.5 Sass (SCSS)



FIGURE 3.19 – Logo de Sass

Sass (Syntactically Awesome Style Sheets) a été adopté comme préprocesseur CSS pour la partie cliente de notre application. Intégré nativement dans notre projet Angular via l'extension `.scss`, il nous a permis de pallier les limitations du CSS classique. L'utilisation de variables, de *mixins* et de l'imbrication (*nesting*) a grandement facilité la personnalisation des composants graphiques de *PrimeNG*. Cet outil a garanti le respect de la charte graphique de la plateforme KAFA'AT EMS, tout en produisant un code de style modulaire, réutilisable et hautement maintenable.

3.2.3 Frameworks de tests et Assurance Qualité

3.2.3.1 JUnit



FIGURE 3.20 – Logo de JUnit

JUnit a été utilisé comme framework de référence pour l'écriture et l'exécution des tests unitaires de notre backend Spring Boot. Il nous a permis de valider rigoureusement la logique métier et les algorithmes de calcul complexes des modules *Invoicing* et *Worktime*. Grâce à son riche écosystème d'annotations et d'assertions, nous avons pu garantir la fiabilité du code serveur et prévenir les régressions lors des phases de refactoring.

3.2.3.2 Mockito



FIGURE 3.21 – Logo de Mockito

En complément de JUnit, Mockito s'est avéré indispensable pour isoler le code en cours de test. Ce framework de *mocking* nous a permis de simuler le comportement des dépendances externes (telles que les *Repositories* de la base de données ou les appels d'API tierces). En créant des objets factices (*mocks*), nous avons pu exécuter des tests unitaires rapides, déterministes et totalement indépendants de l'état de notre base de données PostgreSQL.

3.2.3.3 Jest



FIGURE 3.22 – Logo de Jest

Pour garantir la qualité de la partie cliente (Angular), nous avons intégré Jest comme environnement de test. Préféré pour sa rapidité d'exécution et sa configuration minimale, Jest a facilité les tests de nos composants graphiques (*UI*) et de nos services TypeScript. Ses capacités de simulation avancées et ses rapports de couverture de code ont grandement contribué à maintenir la stabilité et la réactivité de l'interface de la plateforme KAFA'AT EMS.

3.2.4 Outils de gestion de projet et de collaboration

3.2.4.1 Jira Software



FIGURE 3.23 – Logo de Jira Software

Dans le cadre de l'adoption de la méthodologie Agile (Scrum) au sein de l'environnement NTT DATA, Jira Software s'est imposé comme l'outil central de notre gestion de projet.

Il nous a permis de structurer le développement des modules *Invoicing* et *Worktime* sous forme de *Sprints* itératifs. Grâce à ses tableaux de bord (Kanban/Scrum), nous avons pu gérer efficacement le *Backlog* du produit, suivre l'avancement des *User Stories* et coordonner les tâches quotidiennes. Jira a ainsi garanti une visibilité totale sur le cycle de vie des fonctionnalités et a assuré une collaboration transparente et continue avec l'équipe d'encadrement.

3.2.4.2 Confluence



FIGURE 3.24 – Logo de Confluence

En parfaite complémentarité avec Jira, Confluence a été exploité comme espace de travail centralisé pour la gestion documentaire de notre projet au sein de l'environnement NTT DATA.

Cet outil collaboratif nous a permis de capitaliser les connaissances, de rédiger les spécifications techniques et fonctionnelles des modules *Invoicing* et *Worktime*, et de maintenir une traçabilité de nos décisions architecturales. Son intégration native avec Jira a garanti une synchronisation fluide entre notre documentation et le suivi Agile de nos *Sprints*.

3.2.5 Contrôle de version et collaboration

3.2.5.1 Git



FIGURE 3.25 – Logo de Git

Git a été adopté comme système de contrôle de version décentralisé pour l'ensemble de notre code source. Cet outil standard nous a permis de suivre minutieusement l'évolution des composants frontend et backend. Grâce à une gestion stricte des branches (*Branching*), nous avons pu développer les nouvelles fonctionnalités (comme la génération des factures ou la gestion des pointages) de manière totalement isolée, évitant ainsi les conflits lors de l'intégration continue.

3.2.5.2 GitLab



FIGURE 3.26 – Logo de GitLab

En parfaite synergie avec Git, GitLab a été utilisé comme plateforme centralisée pour l'hébergement de nos dépôts sécurisés. Au-delà du simple stockage, GitLab a structuré notre flux de travail collaboratif au sein de l'environnement NTT DATA. L'utilisation des *Merge Requests* a grandement facilité les revues de code avec l'encadrement professionnel. De plus, ses fonctionnalités natives de CI/CD se sont avérées indispensables pour l'automatisation de nos pipelines de tests et le déploiement des modules de la plateforme KAFA'AT EMS.

3.3 Présentation des interfaces de l'application

Cette section est consacrée à la présentation des interfaces graphiques (UI) développées pour notre application. Conçues à l'aide du framework Angular et enrichies par la bibliothèque de composants PrimeNG, ces interfaces ont été pensées pour offrir une expérience utilisateur (UX) optimale, ergonomique et parfaitement alignée avec la charte graphique de l'environnement NTT DATA.

Les captures d'écran suivantes illustrent l'aboutissement de notre travail de conception, en mettant en évidence les fonctionnalités clés et les interactions offertes aux différents acteurs utilisant les modules *Invoicing* et *Worktime* au sein de la plateforme KAFA'AT EMS.